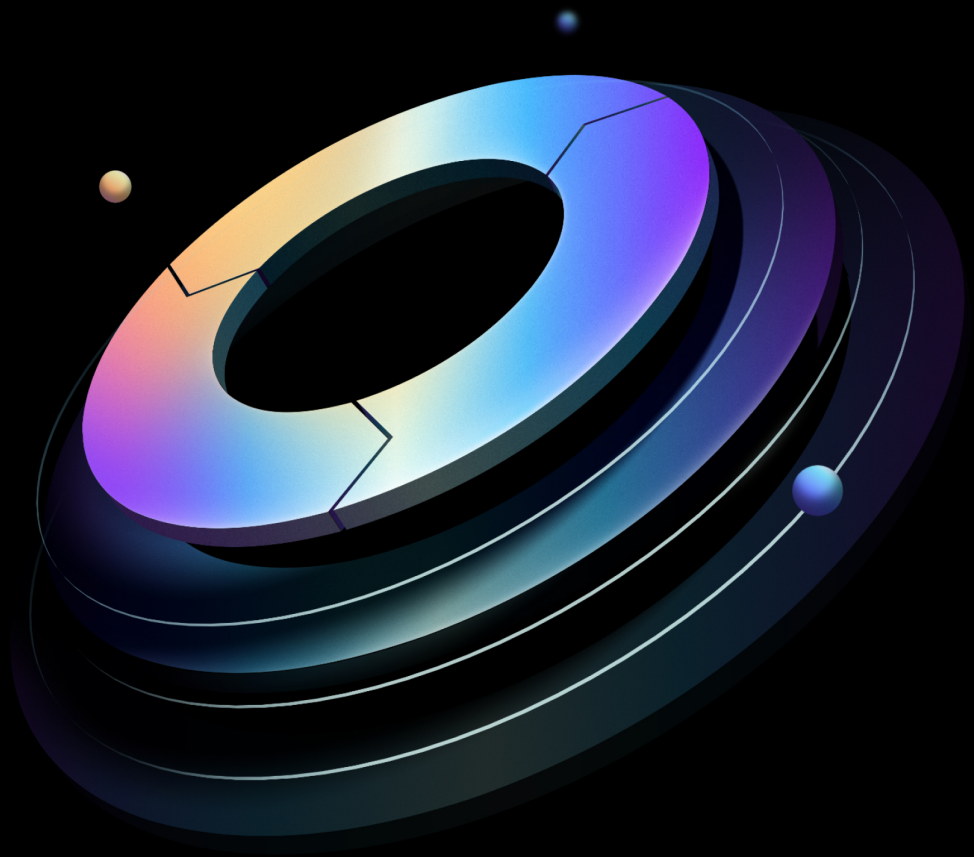
HashiCorp

Boundary Administration Guide

Built from commit 3a6a26c83c

HashiCorp | Validated Designs

09 November 2026



HashiCorp

Contents

1 Introduction **5**
1.1 Why use HashiCorp Validated Designs? . . . 5
1.2 Scope . . . 5
1.3 Prerequisites . . . 5
1.4 Checklist . . . 6
1.5 Language and definitions . . . 6
1.6 What this guide covers . . . 9

2 People and process **10**
2.1 Internal developer platform . . . 10
2.1.1 Producers . . . 12
2.1.2 Consumers . . . 12
2.2 Platform teams . . . 12
2.2.1 Automation tooling teams . . . 13
2.2.2 Golden workflows . . . 14
2.2.3 Consumer workflows . . . 14

3 Integrating identity providers **15**
3.1 Overview . . . 15
3.2 Prerequisites . . . 15
3.3 Roles and responsibilities . . . 16
3.4 Configuring authentication method . . . 17
3.4.1 LDAP/Active Directory auth method . . . 20
3.4.2 OIDC auth method . . . 21
3.4.3 Password auth method . . . 21
3.5 Useful resources . . . 22

4 Managed groups **22**
4.1 Overview of managed groups . . . 22
4.1.1 Centralized identity management . . . 23
4.1.2 Automated group synchronization . . . 23
4.1.3 Scalable access control . . . 23
4.1.4 Reduced administrative overhead . . . 23
4.2 Managed groups implementation and operations . . . 24

4.3 User authentication and authorization workflow with managed groups 25
4.4 Configuring managed groups with the LDAP or OIDC providers 26
4.4.1 Setup LDAP auth method 27
4.4.2 Setup OIDC auth method 27
4.5 Useful resources 27

5 Assigning scopes, principals, roles and grants 27
5.1 Scopes 27
5.2 Assigning principals 28
5.3 Assigning roles to managed groups 29
5.4 Grant permissions to roles 30
5.5 Example scenario 31
5.6 Useful resources 34

6 Automating with Terraform 34

7 Automated target discovery 35
7.1 Overview of host discovery methods 35
7.2 Dynamic host catalogs 35
7.2.1 Dynamic host catalogs workflow with AWS 36
7.2.2 Dynamic host catalogs workflow with Azure 37
7.3 Useful resources 37

8 Administrative governance 37
8.1 Overview 37
8.2 Managing sessions 38
8.2.1 Session initiation 38
8.2.2 Monitoring sessions in real-time 38
8.2.3 Session logging 39
8.2.4 Session recording 40
8.2.5 Session termination 41
8.3 Useful resources 42

9 Audit logs 43
9.1 Event types 43
9.2 Sensitive information 43

9.3 Retention . . . 44
9.4 Configuration . . . 44
9.5 Audit event correlation . . . 45
9.6 Audit log streaming . . . 46

10 Session recording administration 46
10.1 Workers . . . 46
10.2 Storage Considerations . . . 47
10.2.1 Local storage . . . 47
10.2.2 External object storage . . . 48
10.3 Storage providers . . . 48
10.3.1 Amazon S3 . . . 49
10.3.2 MinIO . . . 49
10.3.3 Storage buckets . . . 50
10.4 Lifecycle . . . 51
10.4.1 Storage policies . . . 51
10.4.2 Retention . . . 51
10.4.3 Scope . . . 51

11 Data encryption and key rotation 52
11.1 Overview . . . 52
11.2 KMS providers . . . 52
11.3 Key types . . . 53
11.3.1 External KMS key types . . . 54
11.3.2 DEK key types . . . 55
11.4 Rotating keys . . . 56
11.5 KMS root key migration . . . 56
11.5.1 Updating the “root” key configuration . . . 56
11.5.2 Adding a new root purpose KMS stanza . . . 57

12 Credential store management 58
12.1 Credential . . . 58
12.2 Credential store . . . 59
12.3 Credential library . . . 60
12.4 Architecture . . . 60

13 Vault integration for dynamic credentials 63
13.1 Vault and Boundary for dynamic credentials. 63
13.1.1 Connecting from Boundary to private Vault 63
13.1.2 Boundary organizations, projects, and Vault namespaces. 64
13.1.3 Vault credential TTL vs session TTL 67
13.2 Boundary Credential Store Authentication to Vault 67

14 Approval workflow integration 69
14.1 Integration with approval workflow platforms 69
14.2 How just-in-time approval works with Boundary 72

15 Changelog 74
15.1 2026-08 74

16 Credits 74

1 Introduction

Note

These guides were reorganized in August 2026. The previous Solution Design Guide and Operating Guides (Adoption, Standardization, Scaling) have been replaced with three role-based guide types: Installation, Administration, and User. Read the HVD 2.0 release notes for full details on what changed and where to find the content you need.

1.1 Why use HashiCorp Validated Designs?

HashiCorp Validated Designs (HVD) provide practitioners with opinionated guidance for achieving production-grade deployments of HashiCorp products. These designs are purpose-built for delivering foundational use cases, with a baseline level of architectural and operational maturity. They draw on the field experiences of Solutions Engineers and Solutions Architects working with customers across a wide range of environments and organizational requirements.

Each guide provides access to an opinionated reference architecture, including key design decisions and the rationale behind them. Where applicable, guides identify modular design components that you can adjust to align with organizational or regulatory requirements without compromising the overall integrity of the implementation. For many deployments we include Terraform modules to automate large portions of infrastructure provisioning and software installation.

1.2 Scope

This Administration Guide covers the ongoing, day-2 management of a Boundary Enterprise deployment: identity and access administration, consumer onboarding, observability and alert management, audit log management, and scheduled maintenance such as backups, disaster recovery testing, and upgrades. It assumes the Boundary Installation Guide has been completed and the system is running. For configuring Boundary features and use cases, see the Boundary User Guide.

1.3 Prerequisites

HashiCorp recommends the following prerequisites before using the Boundary Administration Guide:

- Review Cloud Operating Model
- Deploy Boundary Enterprise following the Boundary Installation Guide
- Attend a Boundary workshop

HashiCorp recommends using Terraform to provision and configure Boundary. For guidance on using Terraform, please refer to the Terraform HVD.

1.4 Checklist

Once the Boundary Enterprise deployment is running and handed over from installation, you are ready to implement the day-2 operational practices covered in this guide.

1.5 Language and definitions

While this guide intentionally uses technology-agnostic language, there are some terms that do not translate seamlessly between providers. This document uses the following terms:

Term	Definition
Scope	A permission boundary modeled as a container for resources.
Global scope	The top-level scope that encompasses all child scopes within the boundary system. It serves as the root of the hierarchical structure to organize resources. Configure resources such as storage buckets, storage policies, aliases, workers, users, groups, roles and auth methods at global scope level.
Organization	The intermediate scope level, also referred to as organizations, are a child scope of global. Configure IAM-related resources such as users, groups, roles and auth methods at the organization and global scope levels.
Project	The lowest scope level and a child scope of organization. It allows logical grouping of resources within an organization, such as targets, host catalogs, credentials stores and sessions.

Term	Definition
Host catalog	A collection of hosts that Boundary can connect to, organized into host sets.
Host set	A subset of hosts within a host catalog which are equivalent for the purposes of access control.
Host	A resource with a network address reachable from Boundary, such as a server or database.
Target	A resource that ties network address information (via a direct address or by referencing host sets), credential libraries for injection or brokering (if desired) and port information to represent a networked service available for connection through Boundary. Targets also contain parameters, such as lifetime and connection count, to configure on the sessions created via authorization against the target. A target can be optionally configured with ingress/egress worker filters that determine which workers to use to access targets.
Worker	A secure network proxy, enabling users to access private targets by establishing a direct network tunnel between the Boundary client on the user's machine and the target systems.
User	A resource that represents an individual person or entity for the purposes of access control. It is possible to associate a user with zero or more accounts. A user authenticates to Boundary through an associated account and associated with at least one account before they can access Boundary.
Group	A group is a principal; assign these to roles. Any role assigned to a group is indirectly assigned to all users within the group. Defined a group at the global, organization, or project scope levels.

Term	Definition
Managed group	A resource that represents a collection of accounts. Form the collection by evaluating account information (for example LDAP groups, OIDC claims) defined by the auth method's identity provider against the managed group's configuration. Associate an account with zero or more managed groups within the same auth method. Optionally use these as principals in roles.
Role	A role is a collection of permissions granted to any principal assigned to it. Configure users, groups, and managed groups as principals in a role.
Authentication method	The mechanism by which users authenticate to Boundary via external identity providers such as LDAP, Active Directory, or OIDC providers.
Account	A representation of a user's identity within a specific authentication method. Manually or automatically create accounts and associate them with a user in the same scope as the account's auth method.
Credential	Authentication details such as passwords, tokens, or keys used to access resources.
Credential store	Secure storage for managing and accessing credentials either internally to Boundary or to an external store such as HashiCorp Vault.
Credential library	A collection of credentials of the same type. Broker or inject a single credential library into the network session when users are accessing the networked services via sessions.
Session	A session is a set of related connections between a user and a host. A session may include a set of credentials which define the permissions granted to the user on the host for the duration of the session. Optionally place limits on the session, such as a maximum lifetime and/or a maximum connection count.

Term	Definition
Session recordings	Recordings of user sessions for auditing and monitoring purposes.
Storage Bucket	A container for storing session recordings and other data within Boundary.
Storage Policy	Rules and configurations governing the management and retention of data within storage buckets.
Availability zone (AZ)	A distinct data center within a region that provides redundant and isolated infrastructure to ensure high availability and fail-over protection. Each region consists of multiple AZs to offer resilience against system failures.
Region	A geographically distinct area that hosts multiple data centers, providing redundancy and fault tolerance for cloud services. Each region consists of multiple, isolated locations known as Availability Zones.
Instance	A physical or virtual server or hardware unit used for computing purposes.
Load Balancer	A hardware or software device used to distribute incoming network traffic across multiple servers.

1.6 What this guide covers

This guide covers the day-2 administration of Boundary Enterprise and includes the following:

Section	Summary
People and process	The platform operating model, team responsibilities, and workflows for running Boundary as an internal platform service.
Identity and access management	Ongoing administration of identities, groups, roles, and policies, including operator onboarding and periodic access reviews.

Section	Summary
Consumer onboarding	The runbook for onboarding new consumer teams and applications, including creating isolation boundaries and granting access.
Observability and alert management	Day-2 monitoring: metrics to watch, alert interpretation and response, dashboards, and APM vendor integration.
Audit log management	Day-2 operations for audit logs, including verifying delivery, retention, streaming, and responding to findings.
Backup and restore operations	Verifying scheduled backups, restoring from snapshots, and testing restore procedures.
Disaster recovery operations	Scheduled failover tests, promotion and failback runbooks, and RPO/RTO validation.
Upgrade procedures	Upgrading Boundary controllers and workers with minimal disruption, including pre-checks and rollback.
Support readiness	Preparing for and engaging HashiCorp support, including diagnostics and escalation paths.

2 People and process

Boundary requires thoughtful considerations around people and processes to maximize its value. It is also necessary to rethink how to position and offer an important platform technology such as Boundary. Changing culture and processes can be difficult, as can organizational and political challenges. Our observations across the industry give us a glimpse into what "good" looks like, which we share below. Please keep in mind that no two organizations are the same, and your specific circumstances may require you to choose how you organize your teams.

2.1 Internal developer platform

The Internal developer platform (IDP) has emerged to address the challenge of reducing development cycles and organizational complexity, given cloud computing and IaC have drastically cut

down infrastructure provisioning times.

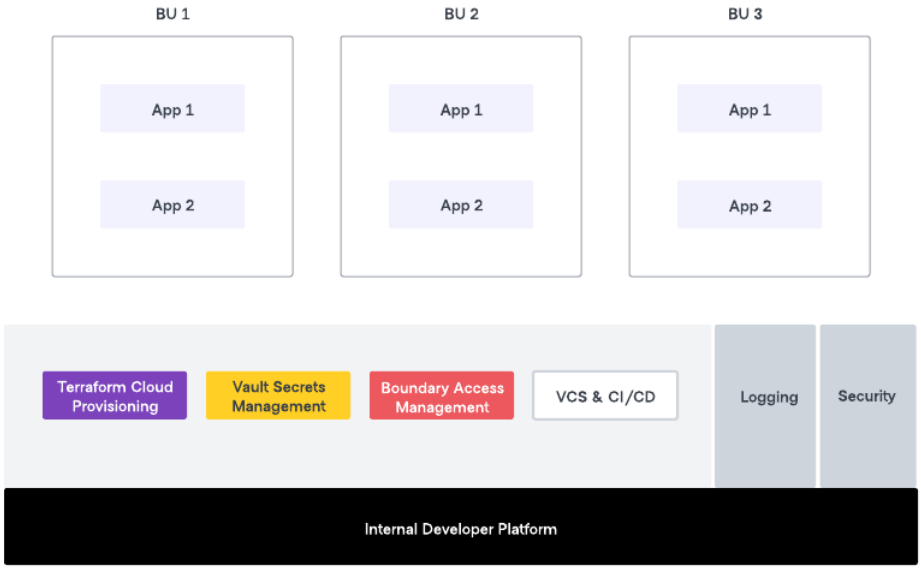


Figure 1: The internal developer platform

The IDP builds on the time-tested shared service model, improved by adding a product management approach. In this model, a Platform team works with developers to build golden paths that development teams can consume in a self-service model.

Access to infrastructure for developers and other stakeholders is a critical component and hence an Access Management service is an integral part of any IDP. We recommend that the team with ownership of the IDP (the Platform Team) also take ownership of operating Boundary, with the Security Team and other related teams providing governance.

With this approach, there are two key roles: producers and consumers. The terms *Producer* and *Consumer* define the roles and responsibilities within a shared service or IDP provided by a Platform team.

Note

While we talk of the *Platform Team* as a singular team, it is possible that multiple sub-teams comprise the term, each with separate areas of responsibility. For example, a separate team may own the access management components of the IDP than that which owns the infrastructure provisioning components. These sub-teams should collaborate to ensure that developers in the various application teams can access these shared services.

2.1.1 Producers

In a shared service model, the producers have a role in configuring the system to meet the needs of the different consumers. Their responsibilities include the following.

- Operating Boundary and ensuring it is available as a mission-critical service.
- Ensuring workflows are well-defined for application teams.
- Set up integration between Boundary and Vault Enterprise for dynamic credentials.
- Work with cross-functional teams to identify targets/hosts to Boundary.
- Enabling consumers.

Producers ensure that the shared services are readily accessible and efficiently utilized by the consumers, empowering them to leverage the benefits of the Boundary platform effectively.

2.1.2 Consumers

In a shared service model, consumers refer to any team within the organization that utilizes the shared services provided by the Platform team. These teams have responsibilities that include:

- Initiating requests for access to targets in Boundary.
- Perform tasks on target systems accessed via Boundary.

Consumers play a crucial role in leveraging the shared services to meet their specific needs and requirements while adhering to the guidelines and standards set by the Platform team.

2.2 Platform teams

Platform teams are responsible for overseeing the overall implementation and management of shared tools and services to enable golden workflows, and driving their adoption within the orga-

nization. When a platform team matures, they tend towards providing their consumers with a user experience, using the IDP, which integrates with implemented golden workflows.

In the context of Boundary adoption, the platform team ensures efficiency and standardization by establishing and enforcing standards and best practices, promoting consistency across project teams and minimizing the risk of errors and inconsistencies that can disrupt operations. They enable scaling and reusability by enabling automated golden workflows for common user workflows that streamline operations and reduce redundant efforts. These golden workflows include and are not limited to the setting up of Boundary organizations, projects, hosts and host sets (including dynamic host catalogs), credential libraries, credential stores and integrating them with Vault Enterprise for dynamic credentials.

By providing valuable Boundary knowledge and expertise to the consumer community, they serve as a centralized resource for expert guidance. This reduces the learning curve for project teams and ensures consistent application of knowledge throughout the organization.

Platform teams facilitate automation and integration, automating common tasks using IaC and integrating it with other DevOps tools. This streamlines processes, increases speed, and reduces manual errors. They also address the security aspect by ensuring adherence to security best practices in infrastructure setup and maintenance, reducing vulnerabilities to cyber threats.

Platform teams also reduce the time to market for new features and applications. With a mature IaC/Terraform service, they expedite the deployment of new infrastructure, management of the various components of Boundary, enabling faster time to market.

2.2.1 Automation tooling teams

Automation tooling teams are a tactical component of the Platform Team. They manage and maintain the automation and tools required for systems and services to operate effectively. They enable and implement golden workflows for consumers allowing them to use the shared services and tools that are part of the platform effectively. As this team matures, they become the product owners and implementers of the IDP.

Automation tooling teams may comprise multiple teams, each responsible for a different tool or group of tools. The CCoE provides overall strategic guidance for this team, and they collaborate with the consumer community to ensure that the services or workflows they enable meet their needs.

In simpler terms, automation tooling teams are responsible for making sure that the tools and automation used by the Platform Team are working properly. They also work with the consumer community to make sure that the services and workflows meet their needs.

2.2.2 Golden workflows

A golden workflow is a standardized, repeatable process for completing a specific task or achieving a specific outcome. It is typically well-documented and tested, and designed to be efficient and effective. Golden workflows are often used in software development, IT operations, and other business processes.

There are many benefits to using golden workflows. They can help with the below.

- Improve efficiency and productivity
- Reduce errors and mistakes
- Ensure consistency and quality
- Improve communication and collaboration
- Accelerate on-boarding and training
- Facilitate automation

With Boundary, the Platform Team enables golden workflows for federating identity and enabling secure access to remote target systems. The Platform Team empowers application development and other teams to independently access target systems and perform their tasks bound with a session time-to-live (TTL) and dynamic ephemeral credentials while maintaining centralized management and control. The Platform Team oversees and governs these workflows, ensuring compliance with policies, security requirements, and best practices.

2.2.3 Consumer workflows

The Platform Team enables consumer workflows and provide these to the various application development teams.

- **Developer workflow**: In this workflow the developer can access remote target systems and perform their operations.
- **Target on-boarding workflow**: This foundational workflow involves on-boarding a target to Boundary. This involves attaching credential libraries and credential stores to a target, setting

up automated target discovery workflows if applicable and setting up RBAC policies. Teams use this workflow to onboard newer hosts for secure remote access.
- **Credentials on-boarding workflow**: For dynamic credentials, this involves determining the secrets storage location within HashiCorp Vault Enterprise for a particular credential and ensuring the credential library can consume those secrets. Security teams use this workflow for the governance of HashiCorp Vault Enterprise to setup dynamic credentials.

3 Integrating identity providers

3.1 Overview

Boundary allows you to secure user access to your infrastructure endpoints (for example SSH, RDP, web/HTTPS, databases, `kubectl`, and any other TCP service) based on the user's identity. Before you configure role-based access controls to these endpoints, you must set up a user authentication method in Boundary. We recommend that you integrate Boundary with your existing identity provider to ensure a consistent and secure user authentication process across your organization. Boundary supports LDAP and OIDC, allowing it to integrate with various identity services and providers, including Active Directory, Microsoft Entra ID, Auth0, and Okta.

3.2 Prerequisites

We recommend that you have completed the following steps before implementing the guidance in this document:

1. You have reviewed and implemented the HashiCorp Validated Design (HVD) for Boundary Solution Design
2. You have a running Boundary cluster.
3. You have valid administrator credentials for your Boundary cluster to configure the authentication method.
4. You have your identity provider configuration details to establish a connection to Boundary.
    - LDAP/Active Directory: Server address (URL and port), TLS certificates, binding DN, and password. Please refer to the LDAP auth method attributes documentation for more information.

- OIDC providers: Client ID, client secret, claims scopes, callback and redirect URL. Please refer to the OIDC auth method attributes documentation for more information.

5. You have configured the network firewall to allow Boundary servers to communicate with your identity provider over the required network ports and protocols, described in the Boundary Solution Design HVD.

3.3 Roles and responsibilities

Below is the list of roles and responsibilities related to integrating identity providers and configuring authentication methods in Boundary.

Role	Responsibility
Platform Team	Configures and manages resources in Boundary. In relation to this use case, the platform team manages Boundary organizations, configures the authentication method, and creates and assigns the organization administrator role.
Identity Management Team	**LDAP/Active Directory:** Manages LDAP user credentials and group memberships. Provides credentials to allow Boundary to query and authenticate users. **OIDC**: Manages the OIDC provider and user claims, registers Boundary as a new client with the provider, and provides necessary client credentials to the platform team.
Network Team	Configures network firewall rules to ensure Boundary servers can communicate with your identity providers over the required network ports and protocols. LDAP/Active Directory: Ensure port TCP-389 (LDAP) or TCP-636 (LDAPS) is open and accessible.OIDC: Ensure port TCP-443 (HTTPS) is open for secure communication with OIDC providers.
Security Team	Ensures compliance with organizational security policies and standards.

Role	Responsibility
Boundary Organization Admin	Manages resources in a Boundary organization, configures managed groups leveraging LDAP groups or OIDC user claim attributes, and configures RBAC policies based on managed groups.

3.4 Configuring authentication method

Many organizations choose to implement Boundary as a service, with a central platform team managing operations while development teams utilize Boundary's capabilities. Enabling this model requires a mechanism to isolate groups of resources within a single cluster. As mentioned above, the global scope is the outermost scope and serves as the entry point for initial administration, setup, and management of the organization scopes. Organizations hold IAM-related resources and project scopes. You create one or more project scopes within an org based on your business workflows. We recommend creating a project scope to organize Boundary resources such as host catalogs, targets, credential stores, and access controls by specific products or teams. Consider a scope as a container for resources and permissions. Each scope has a permissions boundary isolates each scope from the next, creating a defined blast radius.

For your users to securely connect to the targets, they need to be both authenticated and authorized. An authentication method is a resource that provides a mechanism for your users to authenticate to Boundary. It contains accounts that link an individual user to a set of credentials and groups. Principals are entities in Boundary that can be users or groups. Boundary assigns capabilities to these. We recommend that you assign users to groups and use these groups as principals in roles for simplified access control management. Roles in Boundary contain zero or more grants that authorize principals to perform specific actions. You define roles within any scope. A role exists only as long as its scope exists; deleting the scope also deletes the role. The diagram below shows how the different IAM components relate to each other in Boundary.

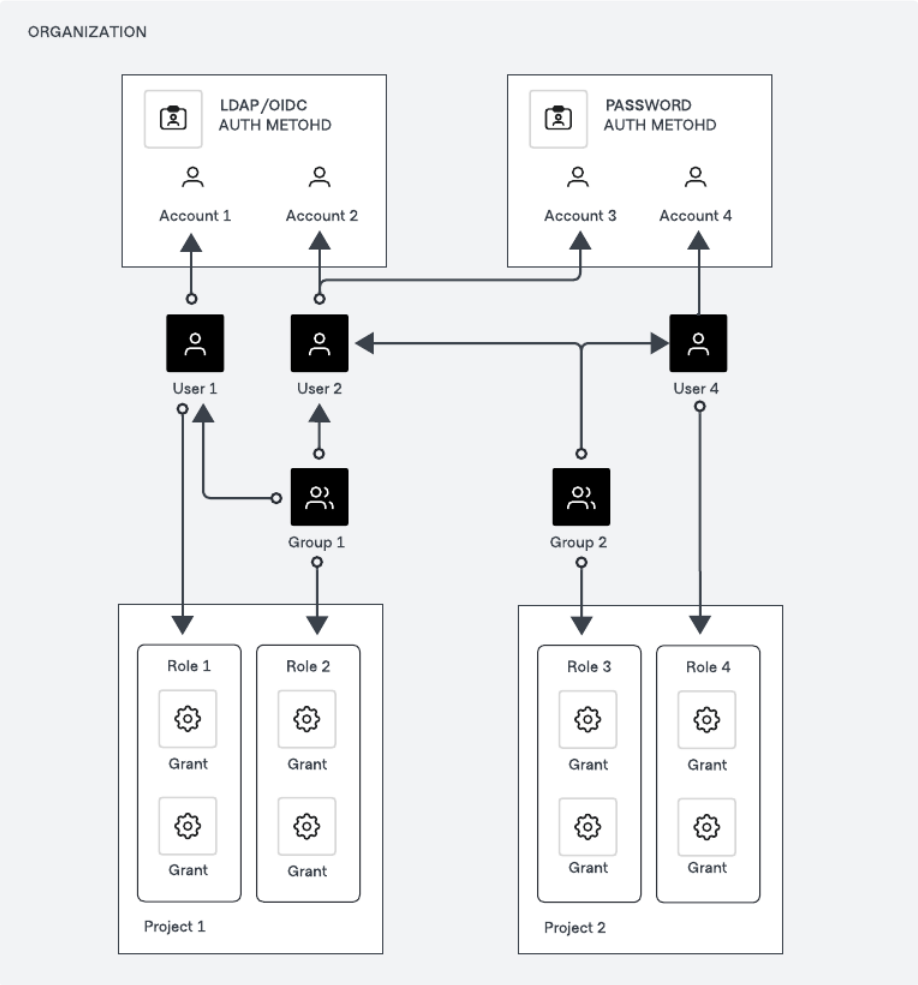


Figure 2: Boundary IAM Model

You can configure authentication methods at either a global or organization scope level. We recommend configuring an authentication method at the global scope level as illustrated in below diagram to provide a consistent authentication experience for all users across different business units. This approach is ideal for enterprise environments where multiple departments or teams use a central authentication system.

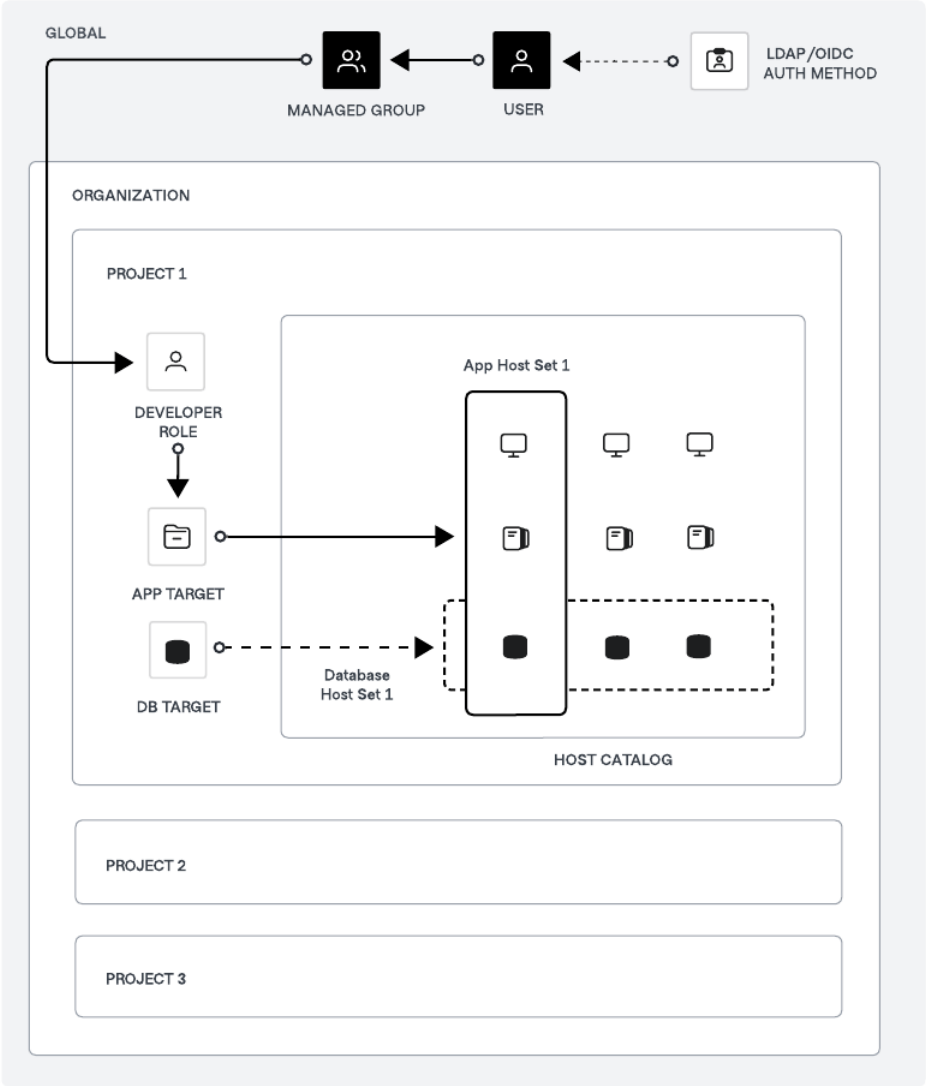


Figure 3: Boundary Auth Method Structure

You can also configure the authentication method at the organization scope level to meet specific requirements. For instance, configure an org-level password authentication method with some accounts for your contractors to provide them access to your resources without integrating these accounts into your primary identity provider, such as Active Directory, Microsoft Entra ID, or Okta.

A user or group assigned to a role at the project scope level can only perform actions on resources within that specific project. For instance, a user assigned the developer role in the preceding illustration can access hosts associated with the app target solely within that project. Consider using an organization scope level role if you need to grant the same permissions to users or groups across multiple projects. Project scope roles control and limit access permissions to specific projects, isolating access and enhancing security. Org scope roles offer broader permissions, simplifying management and ensuring consistency across multiple projects.

Boundary supports several authentication methods, enabling you to leverage your existing identity providers for user authentication. Below are the available authentication methods:

- LDAP
- OIDC
- Password

3.4.1 LDAP/Active Directory auth method

The LDAP auth method allows your users to login using their existing LDAP username and password credentials. The LDAP auth method connects directly to your organizations' LDAP servers. Boundary must be able to communicate with the LDAP server over TCP/UDP-389, or TCP-636 for LDAPS. Please refer to LDAP Authentication for detailed instructions to configure the LDAP auth method.

We recommend you to set the authentication method status to private initially to validate the configuration without exposing it publicly. In this state, the authentication method is not visible to unauthenticated users but can still be used to authenticate users. Once testing is successful, set the authentication method status to public to allow all users to authenticate via LDAP.

When a user makes an authentication request through the auth method, Boundary uses a service account to verify the provided credentials with the backend LDAP server. Users can initiate authentication requests using the Boundary UI, command-line tool, API, or the Boundary desktop app.

The first time a user authenticates using an LDAP auth method, Boundary creates a new account using the user's account login name. If an LDAP auth method enables groups, then each time a user authenticates, Boundary updates their account's group memberships. We recommend that you configure the managed group resource in Boundary to automatically associate the user account to the group based on the LDAP account's associated groups. Managed groups allow you to assign roles within Boundary based on an LDAP account's group memberships.

3.4.2 OIDC auth method

The OIDC auth methods allows Boundary users to authenticate via identity providers such as Okta, Auth0 or Microsoft Entra ID. The OIDC auth method redirects the user's browser to a configured identity provider to complete login. After authentication, the user returns to Boundary's UI, command-line tool, API, or Boundary Desktop. Please refer to the documents below for detailed instructions to configure the authentication method with your identity provider.

- OIDC authentication with Auth0
- OIDC authentication with Okta
- OIDC authentication with Azure

We recommend you to set the authentication method status to private initially to validate the configuration without exposing it publicly. In this state, the authentication method is not visible to unauthenticated users but can still authenticate users. Once testing is successful, set the authentication method status to public to allow all users to authenticate via OIDC.

Users can initiate authentication requests using the Boundary UI, command-line tool, API, or Boundary Desktop. The first time a user authenticates using OIDC provider auth method, Boundary creates a new account using the claims returned from the provider. We recommend that you configure the managed group resource in Boundary to automatically associate the user account to the group based on OIDC account attributes. You can configure managed groups with claims from the JSON Web Token (JWT), or the claims from the UserInfo endpoint.

3.4.3 Password auth method

The password auth method allows you to configure user accounts directly in Boundary.

You can consider password authentication method for the following use-cases:

1. Initial setup and testing
    - Allows you to set up and test Boundary configurations.
    - Ensures Boundary is properly configured and functioning before integrating with other authentication methods like LDAP or OIDC.
2. Backup authentication method
    - Serves as a fallback authentication mechanism if the primary method (for example LDAP or OIDC) fails or is unavailable.
    - You can continue to access and manage Boundary even if external authentication systems experience downtime or connectivity issues.
3. Isolated environments
    - Allows you to authenticate users in isolated or air-gapped environments where external identity providers are inaccessible.
4. Temporary access
    - You can grant temporary access to users, such as contractors or temporary staff, without integrating them into the primary identity provider.

3.5 Useful resources

- Identity management
- Identity and access management (IAM) | Boundary | HashiCorp Developer
- Domain model index | Boundary | HashiCorp Developer
- LDAP auth method attributes
- OIDC auth method attributes

4 Managed groups

4.1 Overview of managed groups

As organizations grow, the administrative tasks of manually managing users and group memberships becomes increasingly challenging. HashiCorp Boundary's managed groups feature provides

significant benefits for enterprises to automatically manage OIDC and LDAP group membership at scale.

4.1.1 Centralized identity management

By integrating with enterprise identity providers (IdP) like Okta, Auth0 via OIDC, or Active Directory via LDAP, Boundary allows enterprises to leverage their existing user directories and group structures. We recommend using the Terraform Boundary Provider for on-boarding and off-boarding of Boundary resources such as users, groups and managed groups, and so on.

4.1.2 Automated group synchronization

Managed groups automatically sync membership based on OIDC group claims from the identity provider (IdP). As your IdP administrator adds or removes users from groups in the central IdP, their access privileges update in Boundary accordingly without any manual intervention.

4.1.3 Scalable access control

With automated group management, enterprises can define role-based access controls (RBAC) in Boundary and map OIDC groups to these roles. By automatically revoking access when users leave the organization or change roles, managed groups help enforce the principle of least privilege and reduce the risk of unauthorized access. Please take note that the synchronization is only updated when users authenticate again to Boundary via OIDC/LDAP to reflect the changes. This allows for granular and consistent permissions management at scale across the entire organization. For off-boarding, when users leave the organization and removed from the IdP group, Boundary automatically removes them from the managed group, reducing the risk of unauthorized access. This process ensures compliance with regulatory requirements and internal security policies by providing audit trails and consistent policy enforcement. Moreover, the reduced need for manual intervention minimizes administrative overhead and potential for human error.

4.1.4 Reduced administrative overhead

By eliminating manual user and group provisioning, managed groups reduce the administrative burden on your teams. This is a particular benefit in large enterprises with thousands of users and

dynamic team structures.

4.2 Managed groups implementation and operations

Boundary organization administrators should consider the following guidelines to implement and operate managed groups.

1. **Integrate with enterprise IdP**: The first step is to configure Boundary to authenticate users with the enterprise's OIDC-compliant identity provider (for example Okta, Auth0, Microsoft Entra ID). Do this with Terraform or through the Boundary UI or command-line tool.
2. **Defined managed group filters**: Boundary admins should create managed groups with filters that select users based on OIDC claims from the IdP or LDAP account's group memberships. For example, a filter like `engineering` in `/userinfo/groups` would automatically add all users from the *engineering* group in the OIDC IdP to the corresponding managed group in Boundary. For LDAP, we can use `group_filter` queries to return groups objects and user objects to resolve group membership according to the structure of your directory schema.
3. **Map managed groups to roles**: Assign managed groups as principals to Boundary roles, which define the specific permissions and access controls for that group of users.
4. **Automate with Infrastructure-as-Code**: Enterprises operating at scale should use Terraform to define and manage Boundary resources, including OIDC auth methods, managed groups, and role mappings. This enables version control, automated deployments, and consistent configuration across multiple Boundary instances.

4.3 User authentication and authorization workflow with managed groups

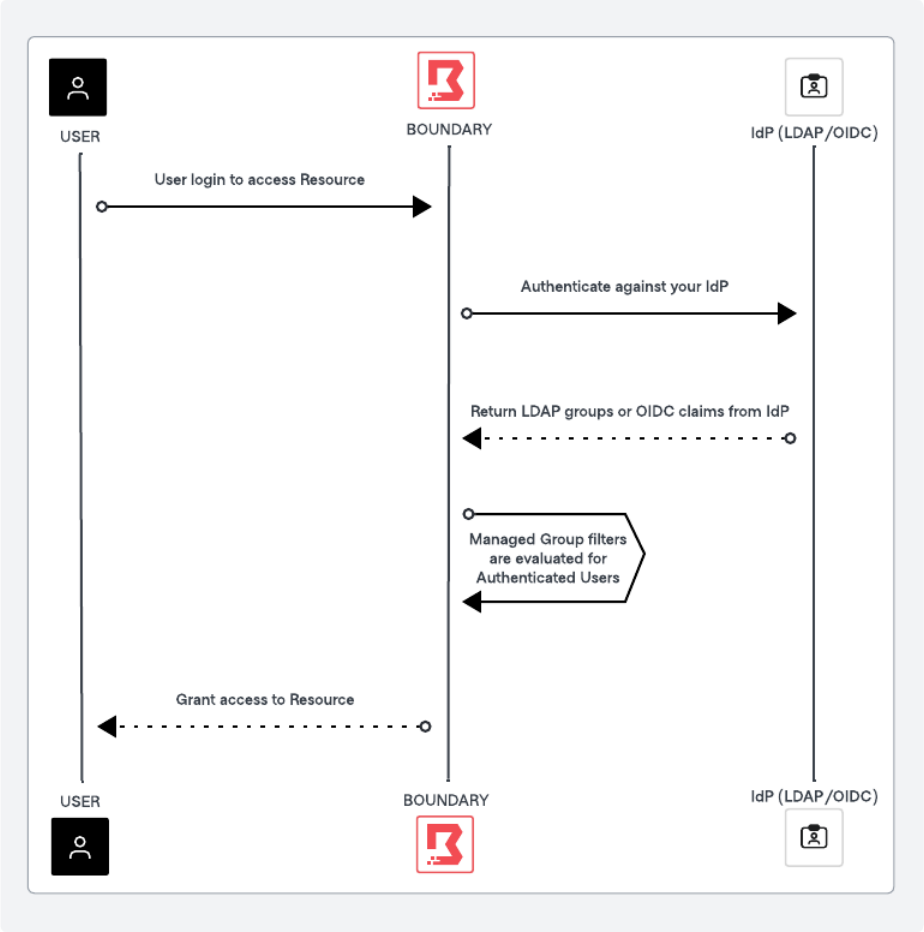


Figure 4: Boundary Managed Group Workflow

The preceding diagram outlines the high level process of user authentication and authorization using Boundary managed groups and IdP providers such as LDAP Active Directory or OIDC providers such as Auth0 or Okta. The key steps involve user authentication via LDAP/OIDC provider, token issuance, claim extraction, managed group evaluation, and role-based access control within Boundary.

1. **User initiates authentication**

- User logs onto Boundary using Boundary Desktop or command-line tool.
- Boundary redirects the user to the LDAP/OIDC provider login page.

1. **User authenticates with LDAP/OIDC provider**

- The user enters their credentials on the LDAP/OIDC provider login page.
- LDAP/OIDC provider verifies the credentials and authenticates the user.

1. **LDAP/OIDC provider issues ID token**

- Upon successful authentication, the LDAP/OIDC provider issues an ID token and redirects the user back to Boundary with the token.

1. **Boundary receives ID token**

- Boundary receives the ID Token from LDAP/OIDC provider.
- Boundary extracts LDAP groups (or) OIDC claims from the ID Token and the UserInfo endpoint.

1. **Evaluate managed group membership**

- Boundary evaluates the user's claims against the managed group filters.
- If the user's claims match the filter criteria, Boundary adds the user to the corresponding managed group.

1. **Assign roles and permissions**

- Managed groups map to specific roles in Boundary.
- The user inherits the permissions associated with the roles of the managed groups they belong to.

1. **Access granted**

- Boundary grants the user access to the requested resource based on the roles' assigned permissions.

4.4 Configuring managed groups with the LDAP or OIDC providers

As a prerequisite, Boundary administrators need to set up managed groups using either LDAP authentication or OIDC authentication. Boundary then uses information from these identity providers to automatically create accounts and users within the same scope as the authentication method.

4.4.1 Setup LDAP auth method

Please refer to LDAP authentication for detailed instructions to configure the LDAP auth method, and an exhaustive list of LDAP auth method attributes configuration parameters.

4.4.2 Setup OIDC auth method

Please refer to the documents below for detailed instructions to configure the authentication method with your identity provider, and an exhaustive list of configuration parameters.

1. OIDC authentication with Auth0
2. OIDC authentication with Okta
3. OIDC authentication with Azure

4.5 Useful resources

- OIDC managed group information and attributes
- LDAP managed group information and attributes
- Filter managed groups
- Managed OIDC IdP groups
- Terraform Patterns for Boundary groups and RBAC

5 Assigning scopes, principals, roles and grants

5.1 Scopes

In Boundary, scopes help in structuring and managing principals. They provide a framework for organizing resources and permissions that enables effective identity and access management. There are three hierarchical levels of scopes.

- Global - the highest-level scope of Boundary where administrators configure and manage Boundary for the entire company.
- Organization - The global scope contains organization scopes and are typically used to represent business units, organizations, or departments. Use organization scopes to also contain multiple production-level auth methods in separate scopes.

- Project - Organization scopes contain project scopes. Use these to separate different business workflows. For example, represent different teams, products, or environments with projects.

5.2 Assigning principals

In Boundary, assign entities roles and grant them permissions, and then refer to these with principals. The main types of principals in Boundary are:

- Users: These are individual accounts representing human users or machines. Associate users with one or more accounts via authentication methods.
- Groups: These are collections of users. A group can include multiple users, allowing for easier management of permissions by assigning roles to the group rather than individual users.
- Managed Groups: Managed groups are typically used in conjunction with identity providers like LDAP or OIDC, which determines membership based on authentication attributes.

Assign these principals to roles, which define the permissions they have within Boundary. A unique ID identifies each principal, and this ID used to assign principals to roles.

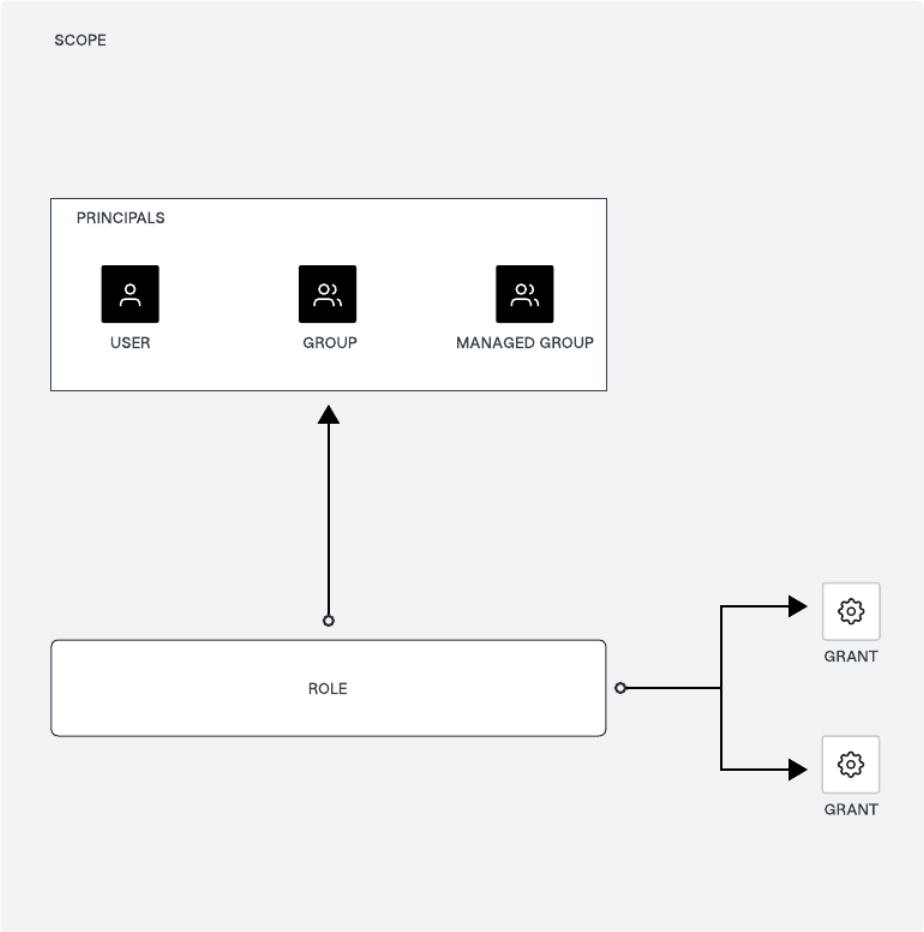


Figure 5: Boundary Principals and Roles Relationship

5.3 Assigning roles to managed groups

We recommend adding a user to a group, and then adding the group to the role(s), instead of adding the user directly to the role(s). This way, you can manage multiple users at the same time, and it is easier to change the permissions of the user by adding/removing them from groups. You can manage OIDC/LDAP users and managed groups the same way, directly in the identity provider.

Roles are collections of capability grants, which are permissions allow roles to take actions and

access resources.

1. Create a role
    - First, create a role within Boundary, defining them at the global, organization, or project scopes. A role contains grants that you assign to principals, including their associated managed groups.
2. Assign principals to the role
    - Second, assign managed groups as principals to that role. Do this by adding the unique ID of the managed group to the role's principal IDs. This step effectively links the managed group to the role, allowing its members to inherit the permissions defined by the role.

5.4 Grant permissions to roles

With the managed group created and automatically managing group membership based on auth methods configured, the next step is to define grants for those principals which are your managed groups.

1. Define grants using *grant strings*, which specify the actions principals can perform on resources. When you define grants permissions, please consider RBAC and *principle of least privilege* (PoLP) in mind. This task adds grant strings to the role. All grants take one of four forms: ID only, type only, pinned ID, or wildcards. The following is an example of a grant string:

    ```
    ids=<id>;type=<type>;actions=<action list>;output_fields=<fields list>
    ```

    - `ids`: Specifies the resource IDs the grant applies to. Use wildcards to apply the grant to all resources of a type.
    - `type`: Specifies the type of resource, such as host-catalog, auth-method, group, etc.
    - `actions`: Specifies the actions allowed on the resources, such as read, list, create, etc.
    - `output_fields`: Specifies which fields of the resource should be visible.
2. Assign grants to the role by specifying the grant strings in the role configuration. The managed group (as a principal) then has permissions to perform the specified actions on the resources.

5.5 Example scenario

Start by creating the following roles at the global scope level for Boundary administrators and Boundary viewers:

Global scope roles:

- `boundary_global_admin` role has the grant of:

```
ids=*;type=*;actions=*
```

- `boundary_global_viewer` role has the grant of:

```
ids=*;type=*;actions=read,list
```

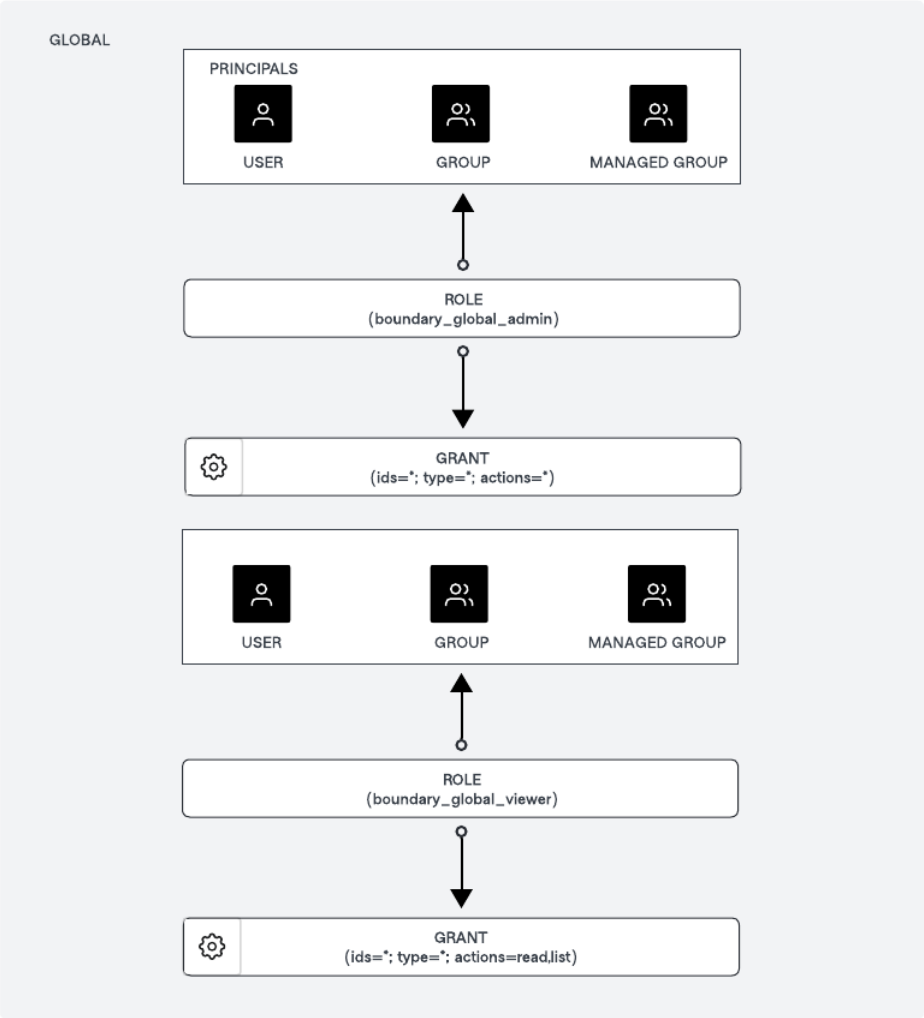


Figure 6: Boundary Principles Roles Grants Global

Next, create a role at the Organization level scope to manage the organization and roles at the Project level scope for multiple projects within an organization, and also the roles to access the targets. (Note: You can scale multiple organizations, and multiple projects based on your organizational structures.)

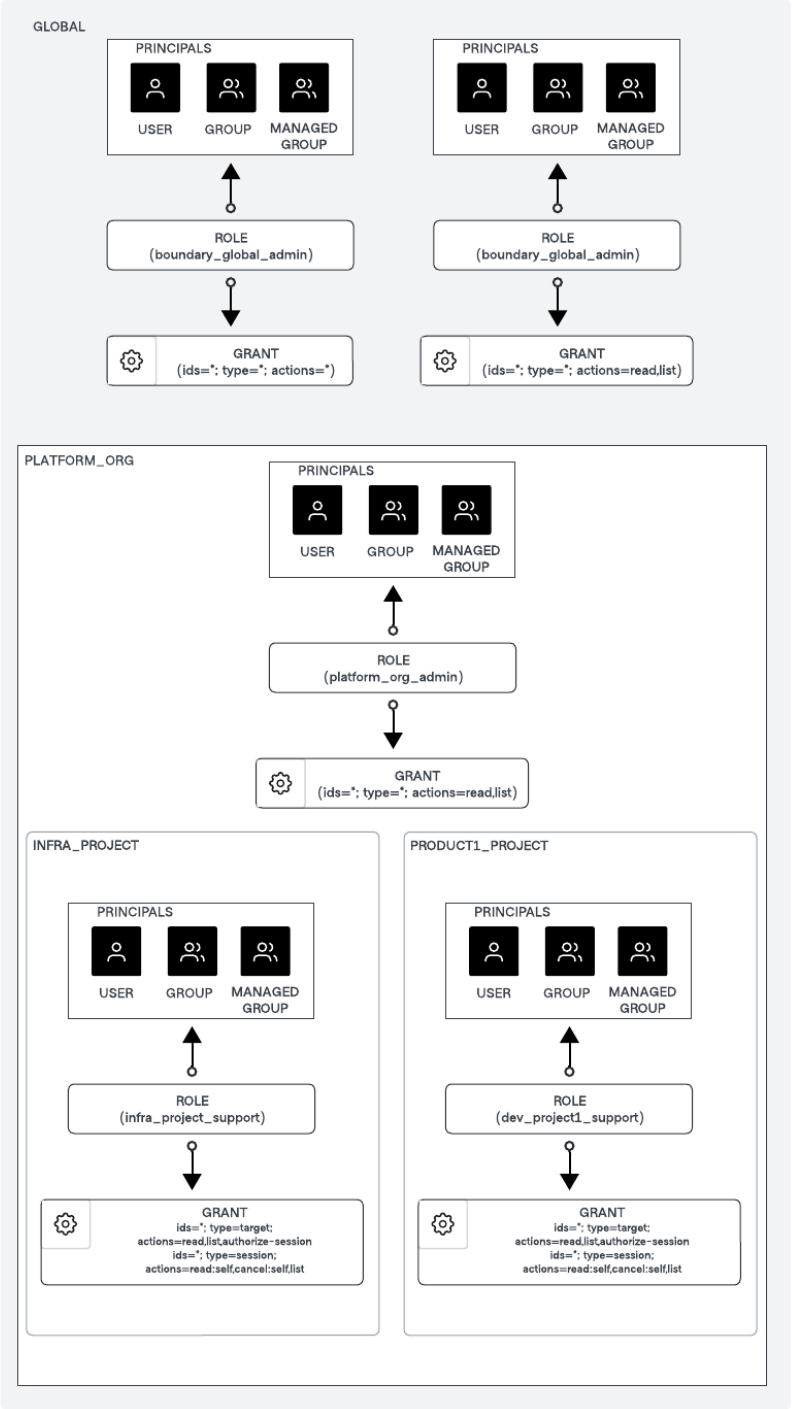


Figure 7: Boundary Principles Roles Grants Organization and Project Scope

Organization scope role for *PLATFORM_ORG*:

- `platform_org_admin` role has the grant of:

```
ids=*;type=*;actions=*
```

Project scope role for INFRA_PROJECT:

- `infra_project_support` role has the grant of:

```
ids=*;type=target;actions=list,read,authorize-session
ids=*;type=session;actions=read:self,cancel:self,list
```

Project scope role for PRODUCT1_PROJECT:

- `dev_project1_support` role has the grant of:

```
ids=*;type=target;actions=list,read,authorize-session
ids=*;type=session;actions=read:self,cancel:self,list
```

5.6 Useful resources

- Refer to Assignable permissions for more information about the permissions you can assign to Boundary principals.
- Refer to Permission grant formats for more information about grant strings and example formats.
- Refer to Manage roles and permissions for instructions to configure roles and grant scopes for principals.
- Refer to the Resource table for a cheat sheet to help you manage your permissions.

6 Automating with Terraform

The first milestones of standardizing your Boundary installation are automation and repeatability. None of your daily tasks should stay forgotten and unaudited because of manual processes.

HashiCorp provides you with the official Boundary Terraform provider for you to be able to configure all the aspects of your Boundary using infrastructure-as-code (IaC) approach.

Most of Boundary Tutorials include Terraform code examples to demonstrate you how typical integrations are made.

Please remember that all the provided Terraform code is not intended to be used as is in your production environment and should only be used for reference.

For details on how to use Terraform to set up Boundary resources and the best practices to be followed, refer to Terraform patterns for Boundary.

7 Automated target discovery

Boundary administrators are responsible for configuring and managing host discovery workflows. They use various methods to ensure hosts and targets are accurately discovered and configured within Boundary.

7.1 Overview of host discovery methods

Boundary supports three primary workflows for target/host discovery:

1. Manual configuration: Administrators manually configure static hosts and targets using the administrator GUI and CLI. This method requires knowledge of the IP address or endpoint used to connect to a host.
2. Configuration-as-code with Terraform: Boundary integrates with Terraform to automate the discovery and configuration of new infrastructure targets. This allows for dynamic configuration without prior knowledge of the target's connection information.
3. Dynamic host catalogs: Boundary automates the ingestion of computing instances and resources from infrastructure providers. Boundary currently supports a dynamic host catalog for AWS and Azure, and we will continue to grow this ecosystem to support additional providers. Hosts are automatically created, updated, and added to host sets, reflecting the connection information maintained by these providers.

7.2 Dynamic host catalogs

We recommend that boundary administrators use dynamic host/target catalogs to automate the discovery and configuration of hundreds of instances at a scale where infrastructure resources are

highly dynamic and ephemeral.

A recommendation is to have a proper tagging of your cloud resources based on organization, business units or product/services team, so that Boundary can discover automatically and manage dynamically.

7.2.1 Dynamic host catalogs workflow with AWS

Boundary administrators leverage dynamic host catalogs to discover and configure AWS resources associated with tags based on `tag:Name=Value`.

For example, instances within AWS could be deployed with tag names and values as follows:

- boundary-1-dev
  - service-type:database
  - environment:dev
- boundary-2-dev
  - service-type:database
  - environment:dev
- boundary-3-production
  - service-type:database
  - environment:production
- boundary-4-production
  - service-type:database
  - environment:production

The host set would then be defined using filters that select the discovered hosts for membership based on the tags defined.

```
boundary host-sets create plugin \
  -name database \
  -host-catalog-id $HOST_CATALOG_ID \
  -attr filters=tag:service-type=database
```

We recommend to begin using dynamic host catalogs for AWS by following the respective setup tutorial guides: Dynamic host catalogs on AWS

7.2.2 Dynamic host catalogs workflow with Azure

Boundary administrators leverage dynamic host catalogs to seamlessly discover and configure Azure resources available through Azure Resource Manager (ARM), adding them as Boundary hosts.

We recommend to begin using dynamic host catalogs for Azure by following the respective setup tutorial guides: Dynamic host catalogs on Azure

7.3 Useful resources

- Concepts: AWS dynamic host catalogs
- Concepts: Azure dynamic host catalogs
- Guided Tutorial: Dynamic host catalogs on AWS
- Guided Tutorial: Dynamic host catalogs on Azure

8 Administrative governance

8.1 Overview

Boundary shows which identities access which systems and provides administrative controls to end sessions automatically or manually. Boundary establishes a system of record for your users' access and actions during remote sessions. This capability allows you to maintain security compliance and ensure robust access controls within your environment.

With Boundary, you can:

- **Monitor access**: Track all systems access, ensuring comprehensive visibility into your users' activities.
- **Manage sessions**: Automatically exit sessions based on predefined policies or manually end suspicious or unauthorized sessions, providing immediate response capabilities.
- **Maintain compliance**: Generate audit trails to meet regulatory requirements and security standards.
- **Enforce access controls**: Implement fine-grained access controls and policies to secure your environment against unauthorized access.

8.2 Managing sessions

A session represents a set of connections between a user and a target. A target allows you to define an endpoint with a protocol and default port to establish a session. A session may include a set of credentials which define the permissions granted to the user on the target for the duration of the session.

8.2.1 Session initiation

The session begins when an authorized user requests access to a target. Boundary sets the expiration time and connection limit for the session if you have configured these attributes on the target. The default session duration is 8 hours (28,800 seconds), after which, Boundary closes all associated connections, and the session exits. Boundary retrieves and returns credentials for each associated credential library. Boundary creates sessions in the project scope of the corresponding target. Deleting a project results in the deletion of all of the active sessions in the project.

8.2.2 Monitoring sessions in real-time

You can view active sessions in real-time, including details like the user's identity, the targets they are accessing, the session start time and the current session status (that is, `active`, `pending`, `canceling` or `terminated`).

You can use the Boundary command-line tool, desktop app or browser based administrator UI to list all sessions.

For example, run the below command to list all sessions across all your projects using the command-line tool.

```
boundary sessions list -scope-id global -recursive
```

To view details of a specific session, use the `boundary sessions read` command with the session ID.

```
boundary sessions read -id <session-id>
```

Similarly, you can use the browser-based administrator UI and navigate to `Sessions` for a given Boundary organization and project to list all sessions.

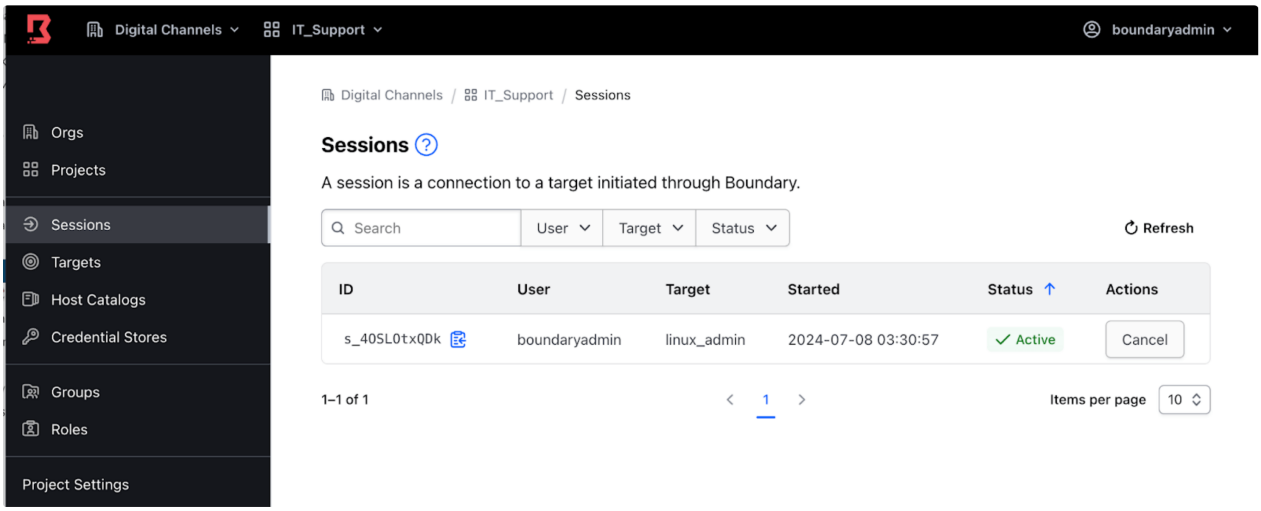


Figure 8: Boundary Sessions Management Active

8.2.3 Session logging

Boundary logs audit events related to user sessions, such as the creation or cancellation of a session. These logs capture critical details including user's identity, session start and end times, and resources accessed. Audit logs allow you to track user activity and enable security teams to ensure compliance in accordance with regulatory requirements.

HCP Boundary supports near real-time streaming of audit events to supported providers, including Datadog and AWS CloudWatch. This feature ensures that security teams have immediate visibility into user activities and potential security incidents. For self-managed Boundary Enterprise, we recommend streaming audit events to your existing centralized logging solution using log shippers. This approach integrates Boundary's audit logging with the organization's existing monitoring and alerting infrastructure. The example below demonstrates an audit event captured when a user initiates a new session to a remote host:

```
{
    "session_id:s_wYID78DBFL"
    "target_id:tssh_N5r14ExLV7"
    "scope:id:p_12BUBsbRog"
    "scope:type:project"
    "scope:name:IT_Support"
    "scope:description:IT Support"
    "scope:parent_scope_id:o_QljIK3QKUc"
    "created_time:seconds:1720352717"
    "created_time:nanos:940476000"
    "user_id:u_SpwJO5YyPh"
    "host_set_id:hsst_I25uYGYFOM"
    "host_id:hst_JCTxpHCzQ2"
    "type:ssh"
    "authorization_token:[REDACTED]"
    "endpoint:ssh://10.200.20.213:22"
    "endpoint_port:22"
    "expiration:seconds:1720381517"
    "expiration:nanos:931225000"
}
```


Please refer to the audit logging section for more details.

8.2.4 Session recording

Boundary also provides auditing capabilities via session recording which is useful for high-security environments where monitoring user actions is critical for regulatory and compliance. Boundary associates a session recording with a target. The session recording captures all interactions that take place during the session, including metadata about the user, target, and any hosts, host sets, host catalogs, or credentials used. A session recording represents a directory structure of files in an external object store that together are the recording of a single session between a user and a target.

Boundary workers record sessions. Workers are the proxy between an end user and a target. A session recording represents connections as separate entities within the recording. Each recorded connection may also contain a recorded channel. This represents a single channel in which the user interacts with the target in protocols that multiplex user interactions over a single connection. For example, the SSH protocol multiplexes user interactions in a single connection, so Boundary records a user's interactions over SSH in a channel.

You can replay recorded sessions through the Boundary administrator UI. This feature allows you to

review user actions and investigate incidents, providing context on user actions during that session. Please refer to the find and view recorded sessions for more details. Define session recordings storage lengths based on organizational policies and compliance requirements.

Note: Only the SSH protocol supports session recording.

8.2.5 Session termination

Boundary enables you to manage and exit sessions both automatically and manually. As mentioned, you can view all active sessions in real-time using the command-line tool, Boundary Desktop or browser-based Boundary administrator UI.

Manual termination

You can manually close sessions directly from the Boundary administrator UI with a few clicks or using the command-line tool commands. This capability allows you to close sessions if you detect any suspicious or unauthorized activity.

For example, to close a session, navigate to `Sessions` for a given Boundary organization and project in the Boundary administrator UI, and click the `Cancel` button.

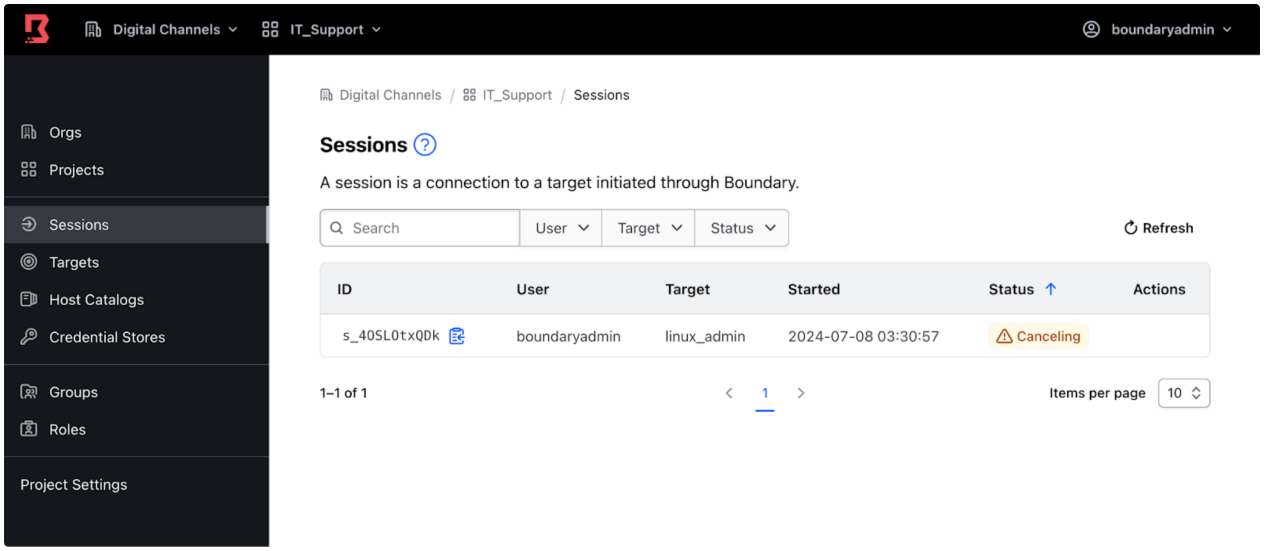


Figure 9: Boundary Sessions Management Cancel

The session status changes to `Canceling`, followed by `Terminated`. At this point, Boundary closes the user's connection to the session's associated target.

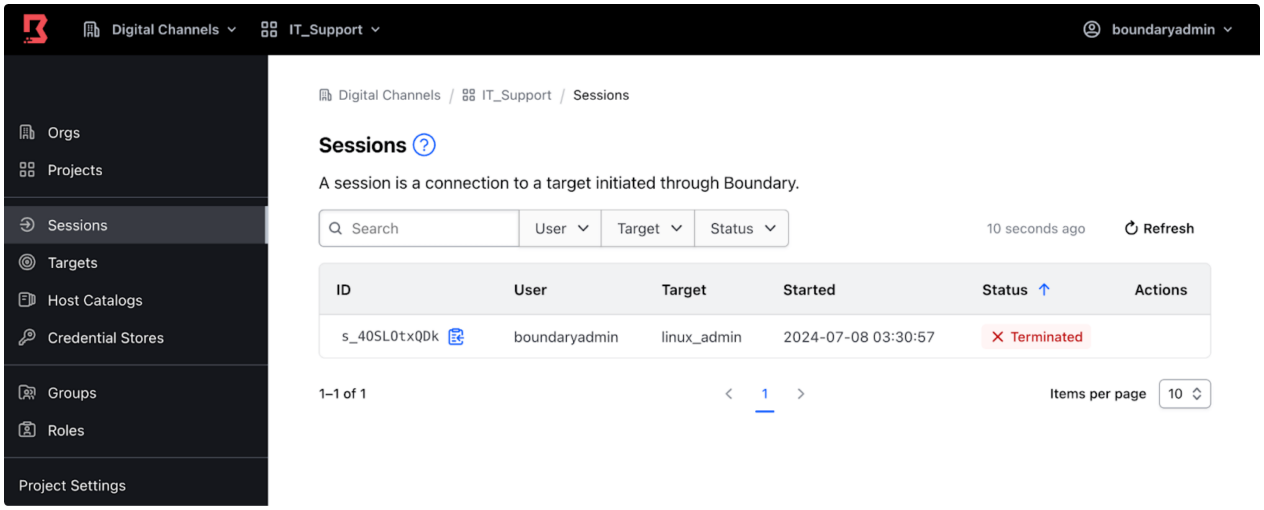


Figure 10: Boundary Sessions Management Terminate

Similarly, to close a specific session using the command-line tool, use the `boundary sessions cancel` command with the session ID.

```
boundary sessions cancel -id <session-id>
```

Automatic termination

You can configure maximum session duration for a target after which the session is automatically terminated. If not configured, Boundary sets the default session duration to 8 hours. This ensures sessions do not remain active indefinitely, reducing the risk of exploitation of unattended sessions. We highly recommend configuring a maximum session duration for a target to limit the time a potentially compromised session remains active, reducing the window for malicious activities. Setting a maximum duration also ensures that unattended sessions are automatically closed, requiring users to re-authenticate to prevent compromise and confirm that access is still valid.

8.3 Useful resources

- Session recording

- Session management

9 Audit logs

An important principle of securing access to sensitive resources is creating a system of record for users' access and actions over remote sessions.

For many organizations, demonstrating compliance with their infrastructure's security posture to internal or external auditors is a critical requirement. In this context, records of remote access are often necessary.

Various laws and regulations impose record-keeping requirements. These stipulations outline the activities that need to be recorded and the duration for which the records must be retained. One of the primary reasons an organization maintains records of system access is to comply with these record-keeping requirements.

By default, Boundary does not emit audit events. Organizations should configure Boundary to emit audit events, which are ingested with the appropriate log analytics platform.

Boundary emits audit events for all requests and responses made to a Boundary controller, every authentication attempt, and all upstream requests made from workers to a controller.

9.1 Event types

There are three types of audit events emitted by Boundary: audit, observation, and error.

- **Audit** will be used for any user action that could contain sensitive material.
- **Observation** is any action that occurs within the execution of an event within the application. For example, any function that is called within the process of an event
- **Error** is utilized to handle any event that had an action that did not occur as expected.

9.2 Sensitive information

Boundary supports sanitizing sensitive information from audit events, and Boundary administrators can configure which sensitive information is encrypted or redacted.

Boundary audit events will support three levels of data redaction: none, hmac-sha256, or encrypted. The default is set to none. If hmac-sha256 or encrypted are specified, then a corresponding KMS for audit must be specified in Boundary's configuration.

Organizations should only sanitize if required by laws, regulations, organizational standards, or policies.

Boundary classifies event data into three categories, "public", "sensitive" or "secret".

- **Public** - Boundary events that capture request information contain fields such as "Id," "Method," and "Path."
- **Sensitive** - Boundary events that capture auth information contain fields such as "UserName" and "UserEmail". By default, sensitive data is encrypted unless audit_filter_overrides is configured. Overrides can be configured to "encrypt", "hmac-sha256" or "redact".
- **Secret** - By default, secret data is redacted unless audit_filter_overrides is configured. Overrides can be configured to "encrypt", "hmac-sha256" or "redact".

9.3 Retention

Audit events should be retained for a minimum period to comply with relevant laws, regulations, organizational standards, and/or policies.

9.4 Configuration

The events stanza configures Boundary events-specific parameters.

Default configuration

If no event stanza is specified then the following default is used. This is not recommended for production scenarios.

```
events {
   audit_enabled = false
   observations\_enabled = true
   sysevents_enabled = true
   telemetry_enabled = false
   sink "stderr" {
      name = "default"
      event_types = ["*"]
      format = "cloudevents-json"
      }
}
```

Minimum configuration**

```
events {
   audit_enabled = true
   observations_enabled = true
   sysevents_enabled = true
   telemetry_enabled = false
   sink "stderr" {
      name = "all-events"
      description = "All events sent to stderr"
      event_types = ["*"]
      format = "hclog-text"
      }
}
```

Example of Audit events configurations can be found in the Tutorial

9.5 Audit event correlation

Boundary supports correlating audit events with other systems to meet traceability requirements. To track and correlate requests and responses across Boundary and other systems, Boundary uses a Correlation Identifier in the form of an X-Correlation-ID header.

Boundary will check for a Correlation Identifier, and if present, it will be included in the event stream as a public field. If not, a random UUIDv4 will be generated as the Correlation Identifier.

As the user can provide the Correlation Identifier, Boundary cannot guarantee its uniqueness.

9.6 Audit log streaming

Audit log streaming is HCP Boundary specific and supports near real-time streaming of audit events to existing customer-managed accounts of supported providers.

Supported providers

- AWS Cloudwatch
- Datadog

10 Session recording administration

> **Note**
>
> This page covers the operator setup and management of session recording. For how users view recordings, see the User Guide: Session recording.

10.1 Workers

Boundary workers perform the recording function. Recording data does not pass through the control plane.

The worker stores the session recording on the local disk during the recording phase and then moves it to the external object store when the session has terminated. Boundary stores recordings in the BSR (Boundary Session Recording) format. During session establishment, the control plane passes any credentials needed to access the external object store to the worker performing the recording. See storage for more information on considerations and provider options.

For targets configured with multi-hop workers, the worker configured to access the external object store records the session. If no worker can access the storage backend, the session is terminated, and an error is returned.

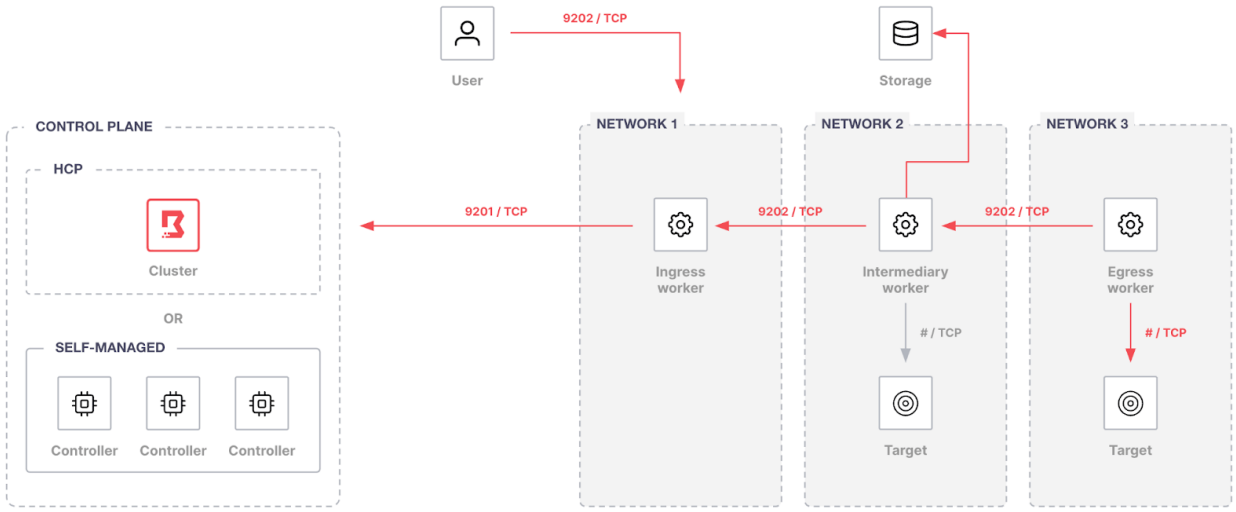


Figure 11: Session Recording

HCP Boundary

Self-managed workers are required to enable session recording with HCP Boundary.

Recorded sessions

Allowed users can view all recorded sessions through a Boundary's administrative web portal.

Please see the tutorial showcasing how to enable session recording with Terraform, AWS and Vault

10.2 Storage Considerations

Before enabling the session recording, one or more storage buckets in Boundary must be created and associated with the external object storage. Organizations must check the considerations for both local storage and external object storage to determine which will meet their requirements.

10.2.1 Local storage

- The number of concurrent sessions that will be recorded on that worker.

- Available disk space is defined by `recording_storage_minimum_available_capacity`
- If organizations don't configure the minimum available storage capacity, Boundary uses the default value of 500MiB.

Refer to the following example configuration to configure workers for session recording storage:

```
worker {
   auth_storage_path = "boundarydemo-worker-1"
   initial_upstreams = ["10.0.0.1"]
   recording_storage_path = "localstoragedirectory"
   recording_storage_minimum_available_capacity = "500MB"
}
```

If a worker is in an unhealthy local storage state, Boundary does not allow new session recordings or session recording playback until the worker is in an available local storage state.

Organizations can check the storage state determined by `recording_storage_minimum_available_capacity` :

- Available
- Low storage
- Critically low storage
- Out of storage
- Not configured
- Unknown

10.2.2 External object storage

The retention period is how long a BSR will be retained in the external storage and from the actual size of the recording data perspective, at a minimum, a session recording for a session with one connection requires 8KB of space for BSR keys, checksums, and metadata.

Estimating how much storage is required to allocate to workers and the external storage provider for recordings depends on user activity. You need to consider the number of concurrent sessions that will be recorded on that worker.

10.3 Storage providers

Organizations must associate the Boundary storage bucket with an Amazon S3 or MinIO storage.

10.3.1 Amazon S3

- Contains the bucket name, region, and optional prefix, as well as any credentials needed to access the bucket.
- Can use static or dynamic credentials. Organizations can configure static credentials using an access key and secret key or dynamic credentials using the AWS AssumeRole API. Here is an example IAM Role policy that can be referenced.

```
{
   "Version": "2012-10-17",
   "Statement": [
      {
         "Action": [
            "s3:PutObject",
            "s3:GetObject",
            "s3:GetObjectAttributes",
            "s3:DeleteObject",
            "s3:ListBucket"
         ],
         "Effect": "Allow",
         "Resource": "arn:aws:s3:::session_recording_storage\*",
         "Resource": "arn:aws:s3:::session_recording_storage/foo/bar/zoo/*"
         },
         {
            "Action": [
               "iam:DeleteAccessKey",
               "iam:GetUser",
               "iam:CreateAccessKey"
            ],
            "Effect": "Allow",
            "Resource": "arn:aws:iam::123456789012:user/JohnDoe"
         }
      ]
}
```

10.3.2 MinIO

- Organizations must provide service account access keys when configuring a Boundary storage bucket.
- It must be configured with R/W access. If you use a restricted IAM user policy, the following policy actions must be allowed at a minimum.

```
{
  "Version": "2012-10-17",
  "Statement": [
     {
        "Action": [
          "s3:PutObject",
          "s3:GetObject",
          "s3:GetObjectAttributes",
          "s3:DeleteObject"
        ],
        "Effect": "Allow",
        "Resource": "arn:aws:s3:::test-session-recording-bucket/*"
     },
     {
        "Action": "s3:ListBucket",
        "Effect": "Allow",
        "Resource": "arn:aws:s3:::test-session-recording-bucket"
     }
  ]
}
```

10.3.3 Storage buckets

A resource known as a storage bucket is used to store the session recordings. The storage bucket represents a location in an external object storage. A storage bucket's name is optional, but it must be unique if you define one. Storage buckets can be associated with zero to many targets.

Organizations must configure workers for local storage and a compatible S3 storage endpoint to create a storage bucket in Boundary. Follow the Create a storage bucket procedure for detailed configuration steps.

For Amazon S3, the required fields for creating a storage bucket depend on whether you configured the static or dynamic credentials. Although credentials are stored encrypted in Boundary, by default the AWS plugin attempts to rotate the credentials Organizations provide. The given credentials are used to create a new credential, and the original credential is revoked. After rotation, only Boundary knows the client secret the plugin uses. Refer to the required field as below

- Common
  - Worker filter
  - Disable credential rotation
- Static

    - Access key ID
    - Secret access key
- Dynamic
    - Role ARN
    - Role external ID
    - Role session name
    - Role tags

10.4 Lifecycle

Boundary provides storage policies to manage the lifecycle of the stored recordings, allowing administrators to set retention and auto-deletion dates. This helps ensure that recordings are available and accessible for the desired retention period, ensuring organizations can meet regulatory requirements like HIPAA, SOC 2, etc. Auto-deletion helps to reduce management and storage costs by automatically deleting recordings at the designated time and date.

10.4.1 Storage policies

Boundary storage policies play a crucial role in enforcing compliance with specific laws or regulations. It is important to note that the system does not support an undo action. Therefore, updating the storage policy of a session recording can have immediate and possibly unexpected results, such as the immediate deletion of session recordings.

10.4.2 Retention

Organizations should configure Boundary's retention period in accordance with any regulatory requirement minimums, e.g. SOC 2 at 7 years. Generally, it is good practice to set retention periods in alignment to your organizational policies, with considerations of storage costs.

10.4.3 Scope

A storage policy exists in either the global scope or an org scope. Storage policies created in the global scope can be associated with any org scope. However, a storage policy created in an org

scope can only be associated with that org scope. Any storage policies associated with an org scope are deleted when you delete the org.

Organizations should start with a global-scoped storage policy unless an org scope has specific requirements.

11 Data encryption and key rotation

11.1 Overview

Data encryption and key rotation are essential elements of HashiCorp Boundary's security framework, ensuring the protection and integrity of sensitive information. Boundary employs strong encryption methods to secure data both at rest and in transit. Implementing regular key rotation helps mitigate the risks associated with prolonged key usage, enhancing overall security. Boundary's encryption and key rotation capabilities enable you to comply with industry standards and regulatory requirements, such as GDPR, HIPAA, and PCI-DSS, which mandate stringent data protection measures.

Encryption at rest
Boundary employs strong encryption algorithms to secure data stored within its system. This includes encrypting sensitive information such as access credentials, session recordings, and audit logs. The encryption at rest ensures that even if the underlying storage is compromised, the data remains protected and inaccessible without the proper decryption keys.

Encryption in transit
Boundary ensures that data transmitted between clients, controllers, and workers is encrypted using TLS (Transport Layer Security). This prevents eavesdropping and man-in-the-middle attacks, ensuring that data remains confidential and unaltered during transmission.

11.2 KMS providers

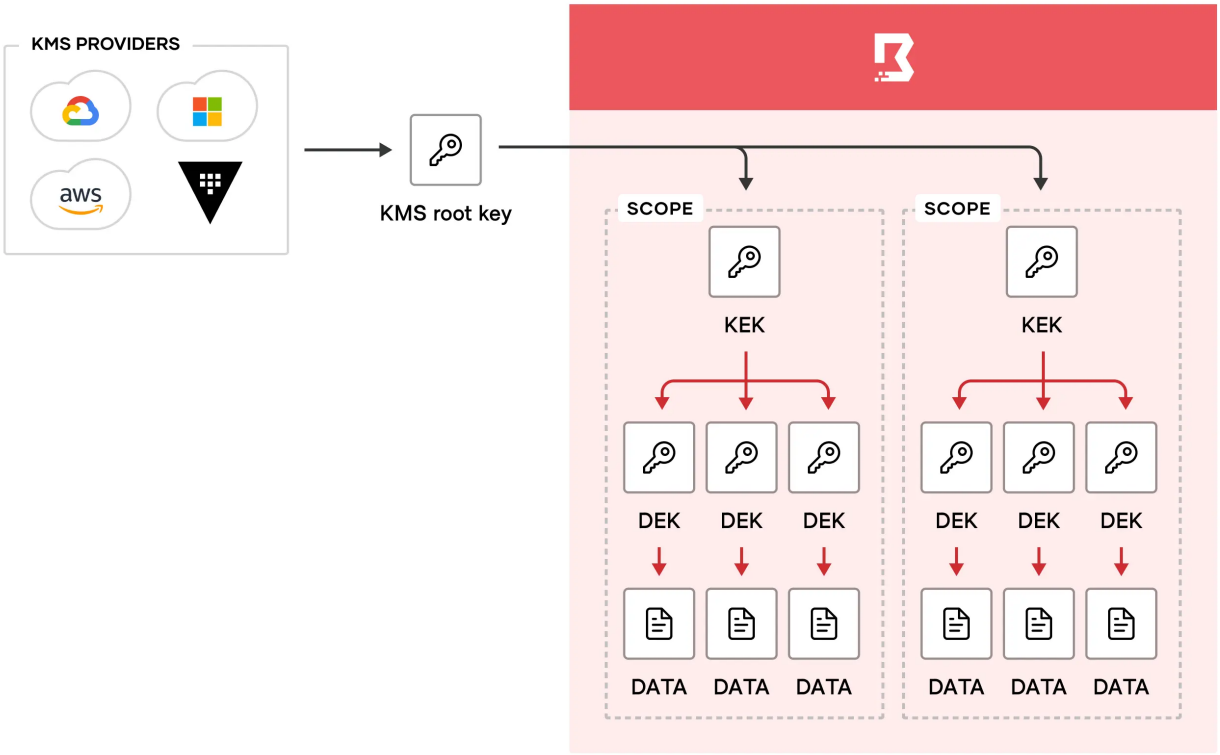
The KMS provider provides the root of trust for keys used in various encryption operations within Boundary, such as encrypting sensitive data stored in the Boundary database or encrypting the data used to authenticate a KMS worker to a controller.

Boundary supports various KMS providers, including HashiCorp Vault, AWS KMS, Azure Key Vault, GCP Cloud KMS, and more. You can configure KMS providers as part of the Boundary controller

configuration. For detailed instructions on configuring KMS providers, please refer to this documentation.

11.3 Key types

The root key provides the root of trust for all scope-specific encrypted data in the Boundary database. This works by having several layers of encryption that all link back to the root key. Whenever you create a new scope, Boundary immediately creates one key-encryption-key (KEK), several data-encryption-keys (DEKs), and a new key version for each of the keys. The key version holds the keying material for the key. Using key versions allows you to rotate keying material, while retaining the same key resource.



Boundary creates following per-scope keys:

- A single key encrypting key (KEK), that is the "root" key for that scope. This key's responsibility is to encrypt the other keys in that scope.
- One or more data encrypting keys (DEKs), each defined for a specific purpose.

Every KEK is encrypted by an external KMS provider. The external KMS is defined as part of the configuration file (potentially with encrypted parameters via a config KMS).

11.3.1 External KMS key types

Boundary uses KMS keys for various purposes, such as protecting secrets, authenticating workers, recovering data, encrypting values in Boundary's configuration, and more. KMS configurations can specify one or more purposes per defined key. These are the currently defined purposes:

- "root": The root KMS key acts as a KEK for the scope-specific KEKs (also referred to as the scope's root key).
- "previous-root": The previous-root KMS key is utilized during the migration to a new root key. Including the previous-root KMS key in your configuration directs the boundary controller to use it for decrypting existing information in the database. This step is essential for rotating and rewrapping the KEKs, thereby completing the migration to the new root key.
- "worker-auth": The worker-auth KMS key is a key shared by the controller and worker in order to authenticate a worker to the controller. If a worker is registered using worker-led or controller-led methods, this is not needed.
- "recovery": The recovery KMS key is used for rescue/recovery operations that can be used by a client to authenticate almost any operation within Boundary.

> Note
>
> It is not required for this kms configuration block to exist in the controller's configuration file. We highly recommend to leave it out except when actually needed, and to use change control capabilities to ensure that the configuration file modification is authorized. After it's no longer needed, the block should be removed.

On the client side, a user can use the `-recovery-config` flag with any operation on the CLI to specify a configuration file containing a suitable kms block. This functionality is also accessible via the Go SDK

Note

Requests authorized via this mechanism show a user of `u_recovery`. This mechanism cannot be used to authorize a session, as there is no uniquely identifying user information available.

- "config": This key can be used to encrypt values within Boundary's configuration file. The config kms block allows you to encrypt sensitive or secret values, such as cloud API keys for KMS's, in boundary's configuration file. This enables you to safely pass the file to a change control system. Only another operator or system with access to that KMS can decrypt the values. Boundary checks for a config KMS block at startup, and if it exists, uses it to decrypt any encrypted values during startup.
- "bsr": The bsr KMS key is required for session recording. If you do not add a bsr key to your controller configuration, you will receive an error when you attempt to enable session recording. The key is used for encrypting data and checking the integrity of recordings.

11.3.2 DEK key types

DEKs are used to encrypt sensitive/secret application data in the database. Boundary encrypts each scope's DEKs with the corresponding scope's root KEK. This KEK is further encrypted using the KMS key designated for the root purpose. This way, only the root of trust (the root key) can be used to decrypt the data in the database, since you need to decrypt the KEK first, which can then be used to decrypt the DEKs, which in turn can be used to decrypt the application data.

The scoped DEKs and their purposes are detailed below:

- audit: This is used to encrypt secret values in the event log. For more information about the event log, refer to the events config.
- database: This is the general-purpose DEK used to encrypt values within the database. Values that are encrypted are those generally considered to be secret, such as API keys, third-party tokens, certificate private keys, and so on.
- oidc: This is used to encrypt OIDC information in cookies and authentication requests.
- oplog: This is used for encrypting oplog (operation log) values for the given scope.
- tokens: This is used for encrypting tokens generated by auth methods within the given scope.
- sessions: This is used as a base key against which to derive session-specific encryption keys.

11.4 Rotating keys

We recommend rotating keys regularly in alignment with security best practices and industry standards such as PCI DSS, which mandate periodic key rotation.

Rotating keys provides several advantages:

- Limiting the amount of data encrypted by the same key version reduces the risk of successful attacks.
- In the event a key is compromised, regular rotation minimizes the number of messages vulnerable to compromise. If there's any suspicion that a key has been compromised, it should be rotated and revoked immediately to mitigate potential damage.
- Regular rotation ensures that Boundary remains resilient and can handle manual rotation effectively during security incidents like key compromise.

When rotating keys, Boundary generates a new key version for the KEK and all DEKs within the specified scope. These new key versions are then used for future encryption operations, while older key versions remain available for decrypting existing data in the database.

Additionally, you have the option to use the rotate key command to rewrap existing key versions with the new KEK version. This process involves re-encrypting both new and existing DEK versions with the new KEK version, ensuring all DEK versions, both new and old, are encrypted with the latest KEK version. This is particularly useful when you want to discontinue using an old KEK version.

For detailed instructions on key rotation and rewrapping, refer to the key version lifecycle management documentation.

11.5 KMS root key migration

There are several reasons to consider migrating from one root key to another. For instance, you might want to transition from an old AWS KMS configuration that was set up with an account you no longer wish to use. Alternatively, you may prefer to migrate to an entirely new KMS provider, potentially opting for a cloud-agnostic solution like HashiCorp Vault.

11.5.1 Updating the "root" key configuration

The first step is to update the existing root purpose KMS stanza to previous-root. This informs Boundary to use this key provided by the KMS for decrypting the existing data in the database. For

instance, if you were using an AEAD KMS provider, the configuration might look like this:

```
kms "aead" {
   purpose   = "root"
   purpose   = "previous-root"
   key_id         = "your-existing-key-id"
   ... // other configuration parameters
}
```

This configuration ensures that Boundary can decrypt the existing database entries using the specified previous-root key during the transition to a new root key.

11.5.2 Adding a new root purpose KMS stanza

Next, add a new root purpose KMS stanza with the new KMS provider configuration and restart the controller. After the restart, Boundary will immediately begin using the new KMS provider for any newly created scopes. However, the existing scopes will still contain KEKs encrypted with the previous root key.

For example, if you are transitioning to a new KMS provider, your configuration might look like this:

```
kms "aead" {
  purpose       = "root"
  key_id        = "new-key-id"
  ... // other configuration parameters
}
```

This configuration ensures that any new scopes created after the controller restarts will use the new KMS provider, while old scopes will continue to function with their KEKs encrypted by the previous root key.

To address this, you need to rotate the keys in all the old scopes, ensuring to specify the rewrap option. This process will re-encrypt all the KEKs with the new root key.

```
$ boundary scopes rotate-keys -scope-id global -rewrap
$ boundary scopes rotate-keys -scope-id <org-id> -rewrap
$ boundary scopes rotate-keys -scope-id <project-id> -rewrap
```

You can now remove the previous-root purpose KMS stanza from the configuration file and restart Boundary again. By doing this, you've successfully migrated from one KMS provider to another.

References:

- Data encryption in Boundary
- KMS configuration
- Boundary KMS (Key Management Service) Root Key Migration

12 Credential store management

> **Note**
>
> This page covers the operator setup and management of credential stores. For how users consume brokered credentials, see the User Guide: Credential management.

When users connect to a remote machine, they typically need a set of credentials for authentication. After they connect to the machine, they may also require another set of credentials to access services or other machines within the network.

In Boundary, there are three main concepts concerning credential management:

- Credential
- Credential Store
- Credential Library

12.1 Credential

A credential is a data structure containing one or more secrets that binds an identity to a set of permissions or capabilities.

A credential:

- Can be of static or dynamic type
- May be a Boundary resource
- Belongs to one and only one credential store
- Can be associated with zero or more targets directly if it is a resource
- Can be associated with zero or more libraries directly if it is a resource
- Is deleted when the credential store or credential library it belongs to is deleted

12.2 Credential store

A credential store is a resource that can retrieve, store, and potentially generate credentials of differing types and access levels. It belongs to a project and supports mechanisms that ensure credentials abide by the principle of least privilege. A credential store can also contain credential libraries.

A credential store:

- Is a Boundary resource
- Can belong to one and only one scope
- Owns zero or more credentials
- Owns zero or more credential libraries
- Is deleted when the scope it belongs to is deleted

In HashiCorp Boundary, credential stores are used to store and manage the credentials required to access various targets. Boundary supports different types of credential stores to accommodate various use cases and credential types.

The primary types of credential stores in Boundary are:

- Static credential stores
- Dynamic credential stores/ Vault credential store

Static credential stores are built into Boundary and only store static credentials (i.e.username and password, ssh-private key and json)

> Note
>
> Credentials stores require an Organization and Project scope to be in place to which they will be associated.

Static credential stores support the following credential types:

- Username password
- SSH private key
- SSH certificate
- JSON

You can find how to configure a static credential store following this guide.

Vault credential stores utilize a HashiCorp Vault instance, which provides additional capabilities such as generating ephemeral credentials.

> **Warning**
>
> While using Terraform to automate your credential stores and static credentials, please be aware that even though Boundary is storing static credentials in an encrypted manner, they are not encrypted by default in your Terraform code and state.

12.3 Credential library

A credential library provides credentials for sessions. All credentials returned by a library must be equivalent from an access control perspective, i.e. any user leveraging these credentials from the same library must have the same access to a target. A credential library manages the lifecycle of the credentials it returns. For dynamic secrets, this includes creation, renewal, and revocation. Rotating credentials include check-out, check-in, and rotation of secrets. The system retrieves credentials from a library for a session and notifies the library when the session has been terminated. A credential library belongs to a single credential store.

Vault credential libraries are the Boundary resource that maps to Vault secrets engines. A single credential store may have multiple types of credential libraries. For example, the Vault credential store might include separate credential libraries corresponding to each Vault secret engine backend.

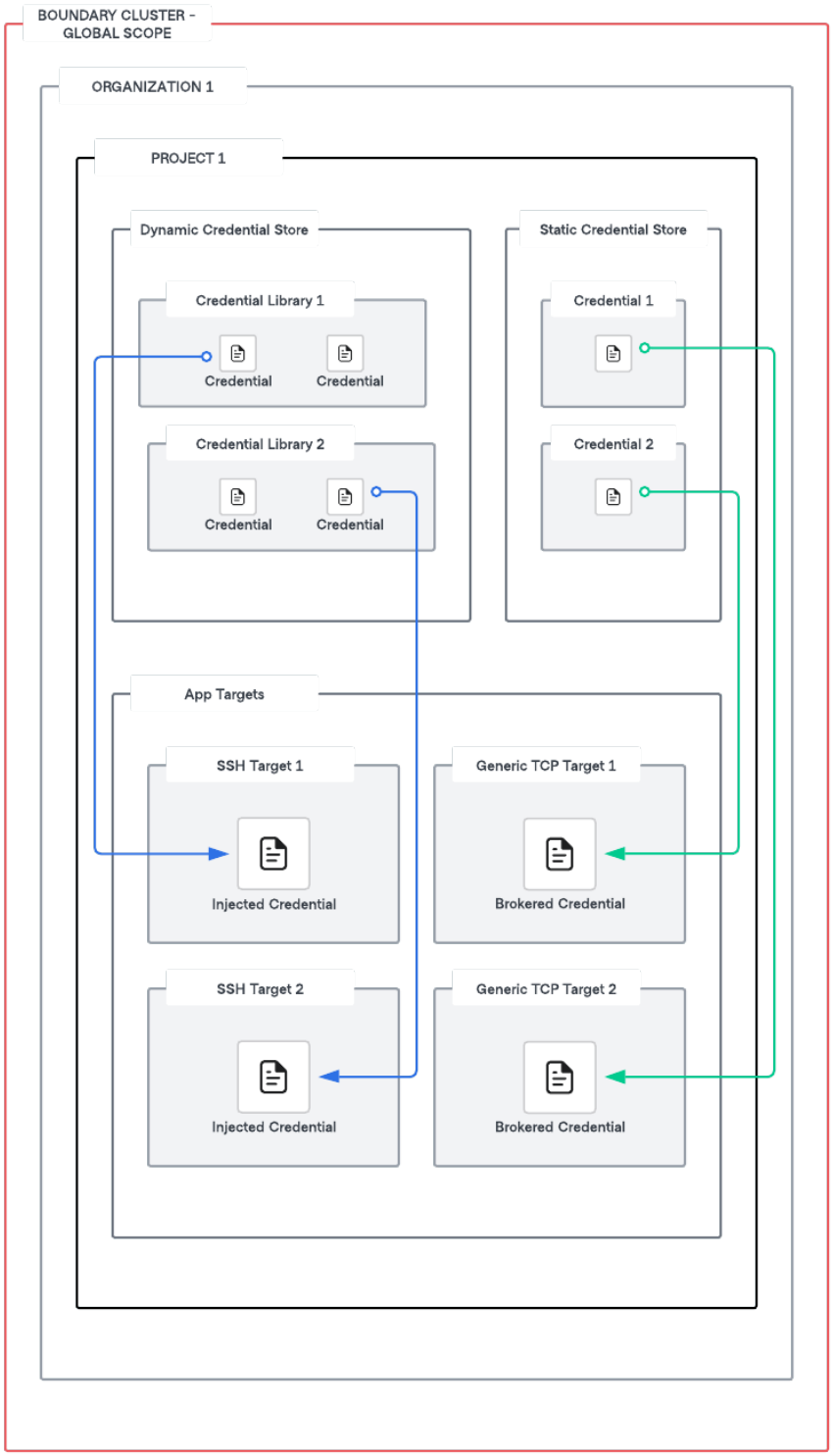
A credential library:

- Is a Boundary resource
- Belongs to one and only one credential store
- Can be associated with zero or more targets
- Can contain zero or more credentials
- Is deleted when the credential store it belongs to is deleted

12.4 Architecture

App target types can be associated with static or dynamic credential stores. SSH target types can have either injected credentials or brokered credentials whereas generic TCP target types can only have brokered credentials. Both dynamic and static credentials can be brokered or injected. More

information about injected credentials and brokered credentials will be discussed in the following section..


BOUNDARY CLUSTER - GLOBAL SCOPE
ORGANIZATION 1
PROJECT 1
Dynamic Credential Store
Credential Library 1
Credential
Credential
Credential Library 2
Credential
Credential
Static Credential Store
Credential 1
Credential 2
App Targets
SSH Target 1
Injected Credential
Generic TCP Target 1
Brokered Credential
SSH Target 2
Injected Credential
Generic TCP Target 2
Brokered Credential

Organizations may have valid reasons for using multiple credential stores within Boundary. You can only create credential stores at the project scope, and each project can contain multiple credential stores. Projects within an org scope may have different requirements from their credential stores. For example, within an org scope, you may have two projects: Database and Compute. These two projects may need to be isolated and can have a dedicated credential store per project.

13 Vault integration for dynamic credentials

> Note
>
> This page covers the operator setup of the Boundary–Vault integration. For how users consume dynamic credentials, see the User Guide: Dynamic credentials.

13.1 Vault and Boundary for dynamic credentials.

HashiCorp recommends utilizing Terraform to provision Boundary clusters (HCP or self-managed), create organizations, and projects, and manage all Boundary resources. The Vault cluster and its resources should be provisioned in the same manner using the Vault Terraform provider).

13.1.1 Connecting from Boundary to private Vault

When connecting HCP Boundary to a private Vault cluster it requires connectivity via an HCP worker. When setting up an HCP worker, it's important to create a worker filter for the credential store. A worker filter will identify workers that should be used as proxies for the new credential store, and ensure these credentials are brokered from the private Vault.

Example worker configuration

```
worker {
   auth_storage_path = "./hcp-worker1"
   tags {
      type = ["worker", "vault"]
   }
}
```

Create a new vault credential store with a worker filter.

Example command for creating a new vault credential store with a worker filter

```
boundary credential-stores create vault -scope-id $PROJECT_ID -vault-address
    $VAULT_ADDR -vault-token $CRED_STORE_TOKEN -worker-filter='"vault" in "/tags /
    type"'
```

In the scenario of self-managed Boundary Enterprise, worker filters may not be required since workers will be provisioned in private networks accessible from Vault.

13.1.2 Boundary organizations, projects, and Vault namespaces.

Vault namespaces are an Vault Enterprise feature that enables isolated Vaults. It provides separate login paths and supports creating and managing data isolated to their namespace. This functionality enables you to provide Vault as a service to tenants.

Everything in Vault is path-based. Each path corresponds to an operation or secret in Vault, and the Vault API endpoints map to these paths; therefore, writing policies configures the permitted operations to specific secret paths. For example, to grant access to manage tokens in the root namespace, the policy path is `auth/token/\`. Managing tokens for a namespace named “education” would be at the following path, `education/auth/token/\`.

Namespace’s support secure multi-tenancy (SMT) within a single Vault Enterprise instance with tenant isolation and administration delegation so Vault administrators can empower delegates to manage their own tenant environment. When you create a namespace, you establish an isolated environment with separate login paths that function as a “mini-Vault” instance within your Vault installation. Users can then create and manage their sensitive data within the confines of that namespace. Reference Vault Namespaces for additional information.

Boundary uses the concept of scopes in its domain model. There are three levels of scopes in Boundary where each scope can have multiple instances of its child scopes

- Global (Highest level scope)
    - Organization (Intermediate level scope)
        - Project (Lowest level scope)

While designing the integration between Boundary and Vault, there are some key factors to consider:

REQUIREMENTS	CONSIDERATIONS
Organizational Structure	What is your organizational structure? What is the level of granularity across lines of businesses (LOBs), divisions, teams, services, and apps that need to be reflected in your Boundary and Vault's end-state design from an organizational perspective?
Self Service Requirements	Given your organizational structure, what is the desired level of self-service required? How are Boundary Credential stores to be segregated and mapped to host catalogs? How are Vault policies to be managed? Will teams need to directly manage policies for their own scope of responsibility, i.e. credential stores?
Audit Requirements	What are the requirements around auditing usage of Vault credentials within your organization?
Secrets engine requirements	What types of secret engines will you use in your credential stores (KV, database, SSH, PKI, etc.)? For large organizations, each of these might require different structuring patterns. For example, with the KV secrets engine, each team might have its own dedicated KV mount.

Considering the above factors, a recommended pattern for mapping Vault Organizations and Namespaces to a Boundary global scope is as follows:

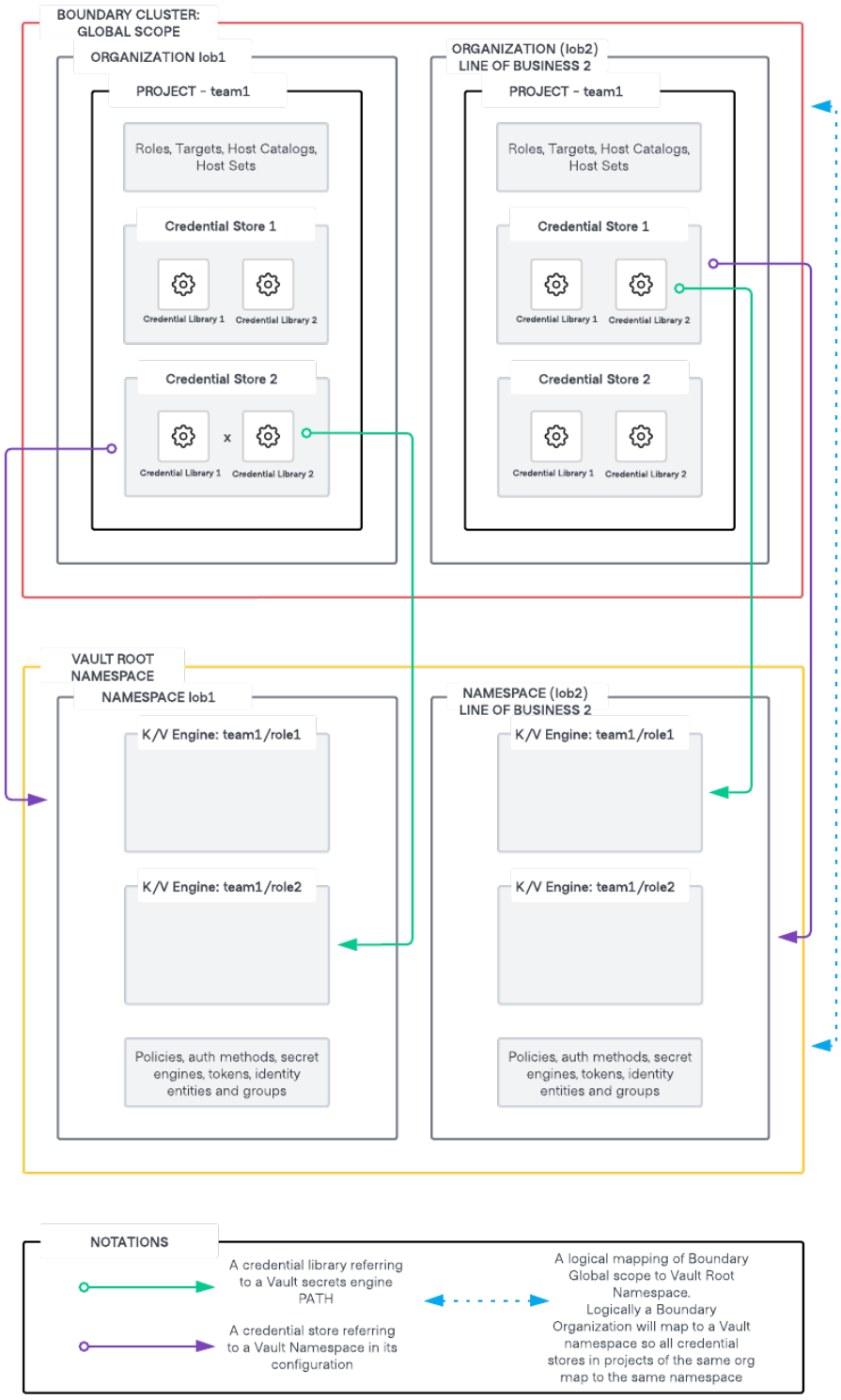


Figure 12: Mapping Boundary Orgs and Projects to Vault Namespaces

- Logically, a Boundary global Scope will map to the Vault root namespace.
- Each line of business [LOB] will have their dedicated organization in Boundary and a dedicated namespace in Vault. Logically an organization in Boundary will map to a namespace in Vault. As you cannot associate Vault's namespace at a Boundary organization scope, hence this mapping is termed as logical.
- All credential stores created in projects that are part of the same organization will refer to the same namespace in Vault for secrets.
  This can further be broken down into hierarchical namespaces within Vault and Boundary credential stores mapping to specific child namespaces within Vault. It is important to keep in mind the principle of least privilege while designing these constructs.
- Credential libraries refer to a particular path in the said Vault namespace. You can have hierarchical paths in secret engines such as Kvv2 (folder structure) to have more granular control over the mapping of specific credential libraries in a credential store to specific paths within the same mount in Vault.

Additional recommendations on Vault namespace design are listed here

13.1.3 Vault credential TTL vs session TTL

Dynamic credentials generated from Vault, such as database credentials, have a time to live (TTL) associated with them. Credentials brokered into a session will forcefully terminate when the TTL for the dynamic credentials expires before the session timeout. If the session timeout is greater than the TTL of the dynamic credentials, the credentials are revoked when the session is terminated.

TTL's are used for static credential sessions to control session termination whereas for dynamic credentials, which are brokered or injected, the credential TTL also dictates the session termination and must be considered while designing.

13.2 Boundary Credential Store Authentication to Vault

Boundary needs to lookup, renew, and revoke tokens and leases to broker credentials properly. This requires these minimum set of permissions for the token created for Boundary authentication to Vault.

- read `auth/token/lookup-self`
- update `auth/token/renew-self`

- update `auth/token/revoke-self`
- update `sys/leases/renew`
- update `sys/leases/revoke`
- update `sys/capabilities-self`

It is important to create policies that give access to the Vault secret engines based on the principle of least privilege. For example, consider a database secrets engine created for managing PostgreSQL credentials. Since Boundary will only be reading credentials from the DB role, read access needs to be provided to only to the particular roles. In this scenario, the following will be the set of permissions required for a database engine mounted at the path `database`

- read `database/creds/<role_name>`

Similarly for connecting to Kubernetes pods using Vault backed credentials, the following will be the set of permissions required for a Kubernetes secrets engine mounted at path `kubernetes`

- update `kubernetes/creds/<role_name>`

Since the Vault token that is generated for Boundary needs to be **periodic, orphan, and renewable**, the following is an example to generate a Vault token based on the above specifications

```
vault token create \
   -no-default-policy=true \
   -policy=<policy1> \
   -policy=<policy2> \
   -orphan=true \
   -period=<period> \
   -renewable=true
```

How to scale this setup? Consider the scenario of multiple secret engines and multiple roles.

- Should we generate an orphan token for each credential store?
- Should we generate an orphan token for each type of database, secret engine, and/or mount and provide the token with permissions for all roles that are part of a mount?
- How should Boundary tokens be scoped?
- Should you create single boundary token for a project or organization?

Considering the multi-tenancy design for Boundary and Vault discussed in the previous section, one of the key considerations to keep in mind is how to manage the scope of the Vault orphan token associated with a Boundary credential store.

It is recommended to create a Vault orphan token in a specific Vault namespace that will be used exclusively by a Boundary project and can have policies attached to read/update the relevant Vault secret engines.

For example, two credential stores in a project that includes a Postgres credential store and an AWS credential store will require two different orphan tokens with their appropriate scopes, respectively.

Breaking this approach to scope orphan tokens to each credential store in a project makes it very difficult to scale as you would be stuck with the overhead of creating and managing orphan tokens for each credential store and managing Vault policies for each one of them which makes it very difficult when you have hundreds of credential stores referencing secret engines in Vault across multiple namespaces.

14 Approval workflow integration

> Note
>
> This page covers the operator setup of approval-workflow integrations. For the user-facing request experience, see the User Guide: Just-in-time approval.

Platform operators integrate Boundary with an external approval workflow platform to enable just-in-time (JIT), time-bound access. This is typically driven by updating IdP-managed groups on approval and expiry.

14.1 Integration with approval workflow platforms

- **PagerDuty:** Integrating with PagerDuty allows for automated incident response workflows. When an incident occurs, necessary personnel can request and receive access to critical systems immediately, ensuring a quick and efficient resolution.
- **ServiceNow:** Service Now is widely used for IT service management. Integrating JIT workflows with ServiceNow allows for access requests to be part of existing ITSM processes, enhancing visibility and control.

- **Slack:** Slack integration allows for real-time communication and approval. Teams can manage and approve access requests directly within their Slack channels, leveraging the collaboration platform for immediate action.

14.2 How just-in-time approval works with Boundary

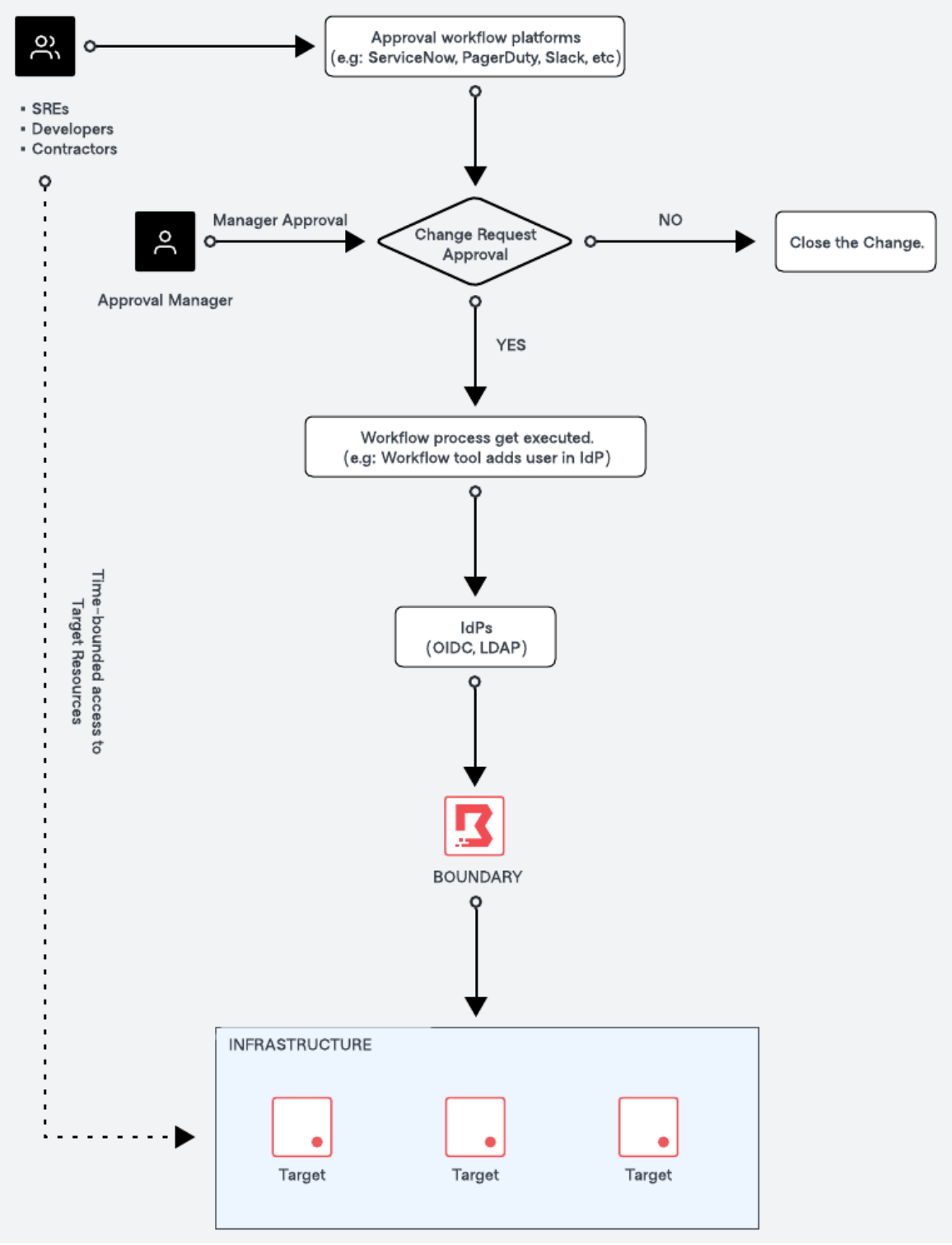


Figure 13: Just-In-Time Flow

The diagram above outlines the high level process of how just-in-time (JIT) approval works with Boundary.

1. Request access submission
    - The user submits a request to access target resources through approval workflow platforms such as ServiceNow, PagerDuty, Slack, etc
    - The request will include the context such as the purpose of the request, the required access time window, and the target resource.
2. Manager approval
    - The manager reviews the change request and makes the decision to either approve or deny the access request.
3. Approval workflow execution (upon approval)
    - Upon approval of the request, the workflow process is executed.
    - E.g: The workflow will update to IdP managed groups (which is our recommended approach) to grant access to the user.
4. Access granted and time-bounded
    - The user is added to the appropriate group within HashiCorp Boundary for a specific time period, and access is granted to the target resource in the specified project. The access granted to the user is time-bounded and will expire after a predefined number of hours.
5. Approval workflow execution (upon expiry)
    - Once the time period expires, the approval workflow is executed again. The flow updates the HashiCorp Boundary configuration to remove the user from the group.
6. Access removal
    - The user's access to the target resource is revoked. And the user is removed from the specific group in the HashiCorp Boundary.
7. Close the change
    - The change request is closed in the approval workflow platform, completing the workflow process.

15 Changelog

This page records notable changes to the Boundary Administration Guide.

15.1 2026-08

- Restructured legacy Boundary HVD content into the HVD 2.0 Boundary Administration Guide as part of the Installation / Administration / User guide migration.

16 Credits

Thank you to everyone who contributed to this validated design guidance. This cross-functional team, representing many departments within HashiCorp, authored, edited, reviewed, and iterated this content to provide prescription and recommendations for using HashiCorp software in enterprise scenarios.

- Andrei Burd
- Danny Knights
- Jeff Mitchell
- Jim Lambert
- Mark Lewis
- Nick Wong
- Ravi Panchal
- Sai Linn Thu
- Shriram Rajaraman
- Theodore Marx
- Tony Phan
- Van Phan